Curso:  Metodología y gestión de proyectos de Ingeniería Software (IS)

Proyecto: EcoLogistics Web (Sistema Inteligente de Gestión de Residuos)

Responsable Académico: José Luis Martínez Vila

Responsable de Ejecución: Angelina Caballero (Rol:Estudiante/Project Manager)

Semana:  0 Año: 2026

# TABLA DE CONTENIDOS

*FASE 1: INICIO Y PLANIFICACIÓN (Gestión)* ..... 1
1. Definición del Product Backlog ..... 1
2. *Selección de Metodología Ágil* ..... 1
3. *Matriz de Asignación* ..... 1
FASE 2: ANÁLISIS Y DISEÑO (Ingeniería CASE) ..... 2
1. Diseño Orientado a Objetos (El "Qué") ..... 2
A- Descripción del Diagrama ..... 3
2. Diseño Estructurado (El "Cómo") ..... 4
A- Diagrama de Flujo del Algoritmo de “Asignación de rutas” ..... 4
B - Explicación del diagrama de Flujo del Algoritmo de “Asignación de rutas” ..... 4

**Semana 0 – Definición y alcance del proyecto**

*FASE 1: INICIO Y PLANIFICACIÓN (Gestión)*
*Objetivo: Organizar el caos antes de empezar.*
*(Ref: ADGG086PO 2.3 Planificación del tiempo, 1.1.4 Metodologías)*
*Tareas del Project Manager*
*Definición del Product Backlog*
*Redactar las Historias de Usuario (Requisitos de los interesados). Ej: "Como conductor, quiero ver la ruta más corta en un mapa".*

1. Definición del Product Backlog
El product Backlog es una lista ordenada de todo lo que el equipo quier construir o mejorar en un producto. Es como una lista de tareas, pero más viva ya que se actualiza, cambia y crece según van apareciendo nuevas ideas, necesidades o problemas relativas al producto. Incluye funciones nuevas, mejoras, correcciones y cualquier trabajo que haga falta para el producto avance.

FASE 1: INICIO Y PLANIFICACIÓN – ECOLOGISTICS WEB
Project Manager: Angelina
Fecha:

Definición del Product Backlog (Historias de Usuario)

Historias de Usuario

- ✓ US01 - Como ciudadano, quiero comunicar una incidencia (como un contenedor dañado o basura acumulada) adjuntando foto y ubicación, para que el ayuntamiento pueda gestionarla con rapidez.
- ✓ US02 - Como ciudadano, quiero recibir un aviso cuando la incidencia que he enviado haya sido resuelta, para confirmar que la actuación se ha llevado a cabo.
- ✓ US03- Como conductor, quiero poder indicar que un contenedor ha sido vaciado, para que el sistema registre su nivel de llenado como 0%.
- ✓ US04 - Como conductor, quiero recibir en mi móvil el itinerario optimizado de la jornada, para reducir el consumo de combustible y aprovechar mejor el tiempo.
- ✓ US05 - Como administrador, quiero asignar manualmente una ruta adicional a un camión cuando se produzca un aumento inesperado de residuos, para evitar que los contenedores se saturen.
- ✓ US06 - Como administrador, quiero gestionar los perfiles de los conductores (crear, eliminar o actualizar), para mantener la información del equipo siempre al día.

*2. Selección de Metodología Ágil*
*Configurar un tablero Kanban (Trello/Jira) para visualizar el flujo de trabajo (To Do / Doing / Done) y establecer las ceremonias de Scrum (Sprints de 2 semanas).*
Para este proyecto vamos a usar un tablero Kanban en Trello con las siguientes columnas:

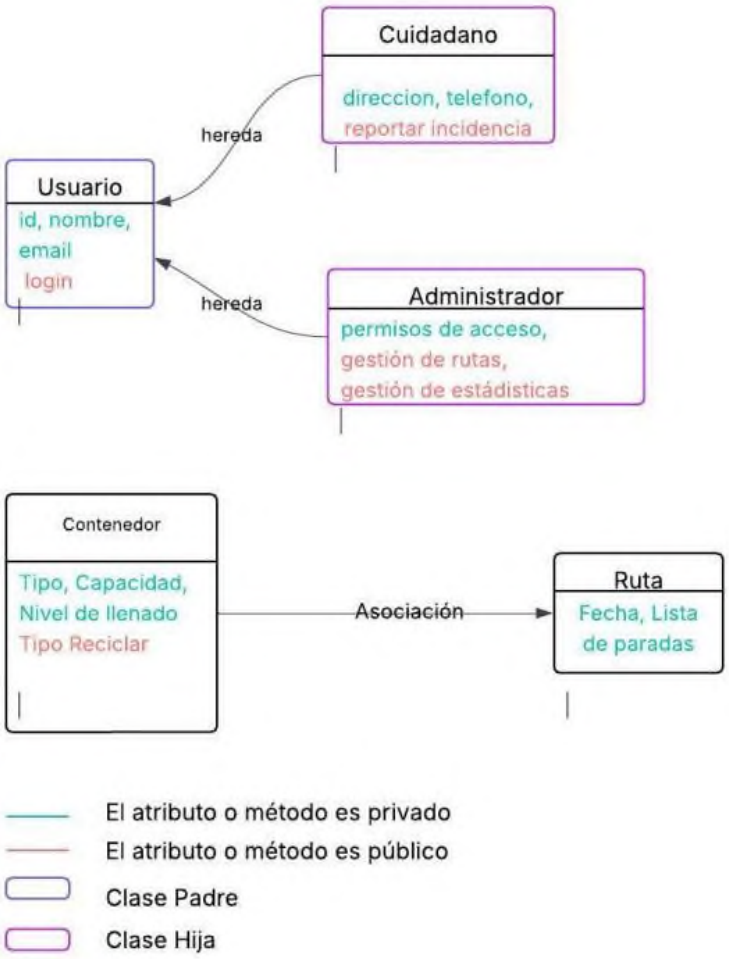
| To Do | In Progress | In Review | Done |
|---|---|---|---|
| | | | |

*3. Matriz de Asignación*
*Decidir qué "empleados" (compañeros) van a cada equipo.*
*¿Pondrás a todos los Seniors juntos o los distribuirás para que enseñen a los Trainees?*
Distribuimos a los 20 empleados (Seniors, 10 Juniors y 6 Trainees) en 4 equipos multidisciplinares o Squads de tal manera que aquellos más preparados o Seniors puedan realizar pair programming con los trainees.

| Squad | Srs. | Jrs. | Trainees | Total | Descripción del trabajo del Squad |
|---|---|---|---|---|---|
| 1 | 1 | 3 | 2 | 6 | Frontend |
| 2 | 1 | 3 | 1 | 5 | Backend |
| 3 | 2 | 3 | 1 | 6 | BBDD & Arquitectura |
| 4 | 1 | 1 | 2 | 4 | Soporte & Tests & Documentación |
| Total | 5 | 10 | 6 | 20 + PM | Incluye Proyect Manager |

**FASE 2: ANÁLISIS Y DISEÑO (Ingeniería CASE)**
**Objetivo: Modelar la solución antes de escribir código.**
**(Ref: 1.1.5 Estructurada, 1.1.6 Orientada a Objetos, 1.1.8 CASE)**
**En esta fase usaremos herramientas Upper-CASE (como StarUML o Lucidchart). Debes entregar dos tipos de diseño:**

**1. Diseño Orientado a Objetos (El "Qué")**
**Debes crear el Diagrama de Clases UML (Ref. 1.1.6).**
**● Tarea: Modelar las entidades.**
**Clase Contenedor (Atributos: tipo, capacidad. Método: getNivelLlenado()).**
**Clase Usuario y sus clases hijas (Herencia): Ciudadano y Administrador.**
**Aplicar Polimorfismo: El método reciclar() actúa diferente si el objeto es Vidrio o Papel**



A- Descripción del Diagrama
El daigrama de clases UML contiene las siguientes Clases y sus correspondientes atributos:

- Clase Usuario (clase padre)

Atributos:
- ID identifica de forma única al usuario, privado
- Nombre es el nombre completo de usuario, privado
- Email es la dirección de correo electrócnico, privado
- Login son las credenciales que dan acceso al sistema si están correctas, público

- Clase Ciudadano (clase hija que hereda de Usuario)

Atributos:
- Dirección es la calle, número, código postal y ciudad del ciudadano,privado
- Teléfono es el número de contacto del ciudadan, privado
- Reportar incidencia es el atributo que permite al ciudadano enviar una incidencia sobre un contenedor, público

- Clase Administrador (clase hija que hereda de Usuario)

Atributos:
- Permisos de acceso es el número que indica el nivel de permiso que tiene el administrador, privado
- Gestión de la ruta es el atributo que permite al administrador crear, modificar o elimininar las rutas de los camiones o conductores, privado
- Gestión de las estadísticas es el atributo que permite crear los informes de los contenedores, las rutas, las incidencias, público

- Clase Contenedor

Atributos:
- Tipo es el atributo que indica cual es el material que recogido por el contenedor, privado
- Capacidad es la cantidad máxima en litros que puede almacenar el contenedor, privado
- Nivel de Llenado es la cantidad que indica el porcentaje (de 0 a 100) de llenado del contenedor, público
- Tipo de reciclado es el atributo que indica como será reciclado el material que se encuentra al interior del contenedor que ha sido recogido.

Este tipo de atributo se considera como un método polímorfico ya que cambia según el tipo de contenedor porque cada material se trata de una manera distinta.
- Si el contenedor es de vidrio, el proceso que el material es distinto ya que hay que calentarlo hasta que se funde y luego darle forma).
- Si el contenedor es de papel, el proceso que el material es diferente ya que aplastarlo, triturarlo y convertirlo en pasta para luego luego volver a fabricar papel de nuevo.

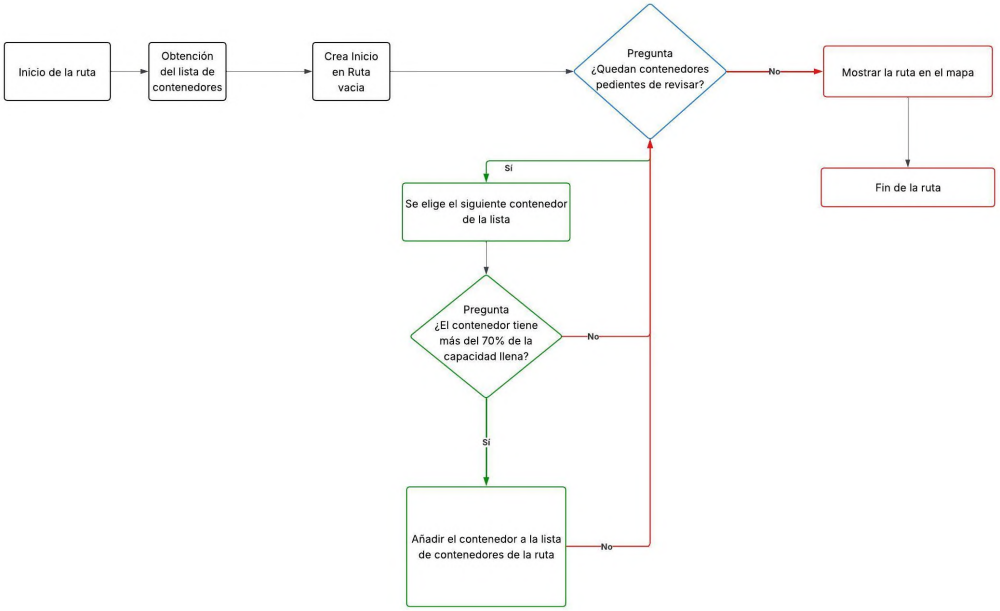
Clase Ruta

Atributos:
- Fecha de la Ruta corresponde a la fecha en la que se efectúa la ruta de recogida, privado
- Lista de Paradas corresponde a una lista que incluye los contenedores que será visitados por el caminión y condutor que realiza la ruta,privado

## 2. Diseño Estructurado (El "Cómo")

**Debes crear el Diagrama de Flujo / Pseudocódigo (Ref. 1.1.5) para los procesos críticos.**
**● Tarea: Diseñar el algoritmo de asignación de rutas.**
**Usar estructuras de Selección (SI nivel > 70% ENTONCES recoger).**
**Usar estructuras de Iteración (MIENTRAS queden contenedores HACER...).**

A- Diagrama de Flujo del Algoritmo de “Asignación de rutas”




B - Explicación del diagrama de Flujo del Algoritmo de “Asignación de rutas”
Paso 1: Inicio de la ruta
En este paso se crea la ruta

Paso 2
En este paso se reunen en una lista todos los contenedores de la cuidad o municipio cuyas rutas se van a crear

Paso 3
Se crea una lista vacía de paradas para ir guardando aquellos contenedores que se vayan recogiendo. Esta lista se va construyendo poco a poco a base de de aquellos contenedores que van superando el nivel de llenado del 70%. Se crea desde cero para evitar que puedan haber errores al incluir contenedores con información de rutas anteriores.

Paso 4
El sistema pregunta si existen contenedores de la lista que no hayan sido evaluados en relación al nivel de llenado (superior al 70% de llenado)

Paso 5
Si todavía quedan contenedores pendientes de ser evaluados pasa a la siguiente fase.

Paso 6
El sistema pregunta si el contenedor tiene un nivel de llenado mayor al 70%

Paso 7
Si su nivel de llenado es es superior al 70% entonces es añadido a la lista de ña ruta de recogida. Y vuelve al paso 4 para seguir  la revisión nuevamente desde este paso otra vez.

Paso 8
En el paso 4 que pregunta si exiten contenedores evaluados es cuando se define la lista de ruta final. Si todos los contenedores son evaluados correctamente se pasa directamente al paso 8.

En este paso se crea el mapa con la ruta del día para el conductor del camión.

Paso 9
Finalización de la creación de la lista de contenedores con el nivel superior al 70% de llenado que crean de la ruta del camión y que será entregada al condutor para su trabajo por parte del sistema en lista y en un mapa.

Curso:  Metodología y gestión de proyectos de Ingeniería Software (IS)

Proyecto: EcoLogistics Web (Sistema Inteligente de Gestión de Residuos)

Responsable Académico: José Luis Martínez Vila

Responsable de Ejecución: Angelina Caballero (Rol:Estudiante/Project Manager)

Semana:  1 Año: 2026

# TABLA DE CONTENIDOS

1. GESTIÓN DE RECURSOS HUMANOS Y ORGANIZACIÓN ..... 1
A - Organización del equipo por Squads ..... 1
B- Organigrama de la Organización del Proyecto EcoLogistics Web ..... 3
*2. ENTORNO DE TRABAJO: INGENIERÍA ASISTIDA POR COMPUTADOR (CASE)* ..... 3
A- Upper-CASE (Análisis y Diseño) ..... 4
a) Herramienta seleccionada: Lucichart ..... 4
b) Herramientas Analizadas para el proyecto ..... 4
B- Lower-CASE (Codificación y Pruebas) ..... 4
a) Lower-CASE seleccionado: Entorno de Desarrollo Integrado - Visual Studio Code 4
b) El Control de Versiones ..... 4
c) Herramientas De Testing Automatizado seleccionadas: ..... 5
C- I-CASE (Integración) ..... 8
a) Herramienta seleccionada: GitHub ..... 8
*3. ARQUITECTURA DEL SOFTWARE: EL ENFOQUE HÍBRIDO* ..... 10
A- Diseño Orientado a Objetos (OOP) ..... 10
a) Clases de Dominio ..... 10
b) Diagrama de Clases del Diseño Orientado a Objetos (OOP) ..... 11
B. Ingeniería Estructurada - Los Procesos Críticos ..... 12
a) Explicación del diagrama de Flujo para el Algoritmo “Asignación de Rutas” ..... 12
b) Diagrama de Flujo para el Algoritmo “Asignación de Rutas” ..... 13
*4. IMPLEMENTACIÓN: INGENIERÍA WEB* ..... 13
*A. Separación Frontend/Backend* ..... 13
a) ¿Por qué es mala idea mezclar HTML con código de base de datos? ..... 13
*B. La "Nube" y la Concurrencia* ..... 14
a) ¿Cómo garantizamos la escalabilidad del sistema EcoLogistics? ..... 14
b) ¿Qué medidas de Seguridad implementar? ..... 15
*C. Tecnologías Web* ..... 15
a) Tecnologías Web - Definición del Stack y Justificación ..... 16
b) Diagrama de Arquitectura Web (navegador – servidor – BBDD) ..... 17

**Semana 1 – PROYECTO INTEGRAL: EcoLogistics Web (Fase 2.0)**
**Documento de Planificación y Arquitectura Técnica**

*PROYECTO INTEGRAL: EcoLogistics Web (Fase 2.0)*
*GESTIÓN DE RECURSOS HUMANOS Y ORGANIZACIÓN*
*(Aplicación de conceptos de Gestión y Metodologías Ágiles)*
*Tú eres el responsable. Tienes 20 empleados (tus compañeros) con los siguientes perfiles:*
*• 5 Expertos (Seniors en BBDD/Backend).*
*• 10 Nivel Medio (Juniors competentes).*
*• 6 Principiantes (Trainees con poca base).*
*Tu Tarea:*
*En tu documento, debes definir cómo organizas estos recursos. No basta con decir "hago grupos". Debes justificar la estructura basándote en la productividad y la transferencia de conocimiento.• Propuesta Requerida: Define los equipos. ¿Crearás equipos especialistas (un equipo solo de BBDD, otro solo de Front) o equipos multidisciplinares (Squads)?*
*• Distribución de Roles: ¿A quién pones a diseñar la BBDD? ¿A quién a maquetar? ¿Cómo usas a los expertos para que los principiantes no bloqueen el proyecto?*
*• Pista: Recuerda el concepto de Pair Programming (visto en XP) o la tutorización técnica.*

1. GESTIÓN DE RECURSOS HUMANOS Y ORGANIZACIÓN
A - Organización del equipo por Squads

Partimos de un equipo formado por 22 personas: 21 empleados (compañeros de clase) y 1 product manager (en este caso Angelina) que actuarán como el equipo de desarrollo y Prouduct Manager.
Estos 21 miembros se dividen en 3 perfiles según su nivel de experiencia:
Perfil A: Seniors (5 personas) se encargan de decisiones de arquitectura, seguridad y algoritmos complicados, también de acompañar y guiar al resto del equipo.
Perfil B: Juniors (10 personas) con conocimientos medios que desarrollan la mayor parte de la aplicación tanto en frontend como en backend.
Perfil C: Trainees (6 personas) con conocimientos básicos que se centran en tareas como maquetación HTML/CSS, documentación, pruebas manuelas y apoyo en la entrada de datos.

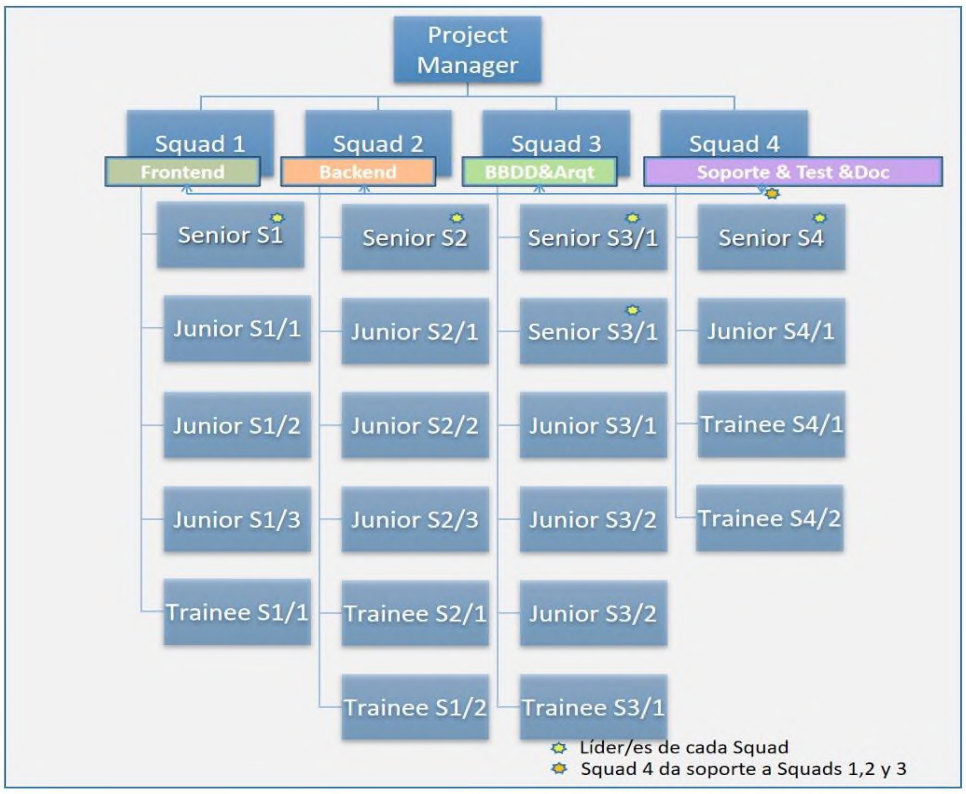
He distribuido a los 3 perfiles en 4 equipos multifuncionales o squads para favorecer el aprendizaje entre perfiles o pair programming, facilitando que el conocimiento fluya y evitar que el trabajo dependa unicamente de unas pocas personas; haciendo que cada equipo sea autónomo y cuente con una combinación equilibrada de experiencia.
De esta forma, los seniors aportan conocimiento y ayuan a los demás, los juniors llevan el peso del desarrollo y los trainees apoyan en tareas más sencillas mientras aprenden.

| Squad 1<br>Este equipo se dedica a trabajar en el Frontend (el interfaz de ciudadano y conductor) | | |
|---|---|---|
| Nombre | Rol | Responsabilidades |
| Senior S1 | Full Stack | Define los estándares de accesibilidad (WCAG) y las guías de diseño.<br>Supervisa la arquitectura del frontend y la optimización del rendimiento visual. |
| Junior S1/1 | Front-end | Implementa componentes reutilizables y accesibles.<br>Garantiza que el diseño sea responsive en distintos dispositivos. |
| Junior S1/2 | Front-end | Desarrolla la lógica de interacción del usuario (formularios, validaciones, navegación).<br>Optimiza tiempos de carga y reduce el uso innecesario de recursos. |

| | | |
|---|---|---|
| Junior S1/3 | Front-end | Construye prototipos interactivos y anima flujos clave (transiciones, feedback visual).<br>Mejora la usabilidad mediante pruebas con usuarios. |
| Trainee S1 | Soporte | Realiza pruebas manuales de interfaz (UI testing).<br>Documenta mejoras de usabilidad y propone ajustes visuales. |
| Squad 2<br>Este equipo se dedica a trabajar en el Backend (la lógica de las rutas, las API) | | |
| Nombre | Rol | Responsabilidades |
| Senior S2 | Full Stack | Diseña la arquitectura de servicios y define las reglas de negocio principales.<br>Supervisa la calidad del código backend y los patrones de diseño. |
| Junior S2/1 | Back-end | Implementa microservicios internos y gestiona la comunicación entre ellos.<br>Optimiza consultas y operaciones intensivas del servidor. |
| Junior S2/2 | Back-end | Desarrolla servicios de autenticación, autorización y gestión de sesiones.<br>Mantiene la lógica de negocio relacionada con usuarios y permisos. |
| Junior S2/3 | Back-end | Implementa servicios de notificaciones (email, push, avisos internos).<br>Gestiona colas de mensajes y tareas asíncronas. |
| Trainee S2/1 | Soporte | Realiza pruebas de API con herramientas como Postman.<br>Documenta endpoints y flujos de negocio. |
| Trainee S2/2 | Soporte | Apoya en la creación de mocks y datos de prueba.<br>Participa en pair programming para aprender patrones backend. |
| Squad 3<br>Este equipo se dedica a gestionar la base de datos, la arquitectura, la seguridad y la calidad de la plataforma | | |
| Nombre | Rol | Responsabilidades |
| Senior S3/1 | DevOps | Diseña la infraestructura en la nube, pipelines CI/CD y monitorización.<br>Define políticas de despliegue y rollback. |
| Senior S3/2 | Seguridad & Calidad | Establece controles de seguridad, cifrado y auditoría.<br>Supervisa la calidad del software mediante métricas y revisiones. |
| Junior S3/1 | DevOps | Automatiza despliegues, backups y tareas de mantenimiento.<br>Configura herramientas de observabilidad (logs, métricas, alertas). |
| Junior S3/2 | Pruebas | Implementa pruebas de carga, estrés y rendimiento.<br>Evalúa la estabilidad del sistema bajo alta demanda. |
| Junior S3/3 | Pruebas | Desarrolla pruebas automatizadas de regresión y smoke tests.<br>Mantiene suites de testing y reporta métricas de calidad. |
| Trainee S3/1 | Soporte | Documenta procesos de infraestructura y seguridad.<br>Ayuda en tareas básicas de mantenimiento. |
| Trainee S3/1 | Soporte | Colabora en la ejecución de pruebas manuales y validación de entornos.<br>Apoya en la gestión de incidencias técnicas. |
| Squad 4<br>Este equipo se dedica a gestionar el mantenimiento, soporte y la documentación de la plataforma | | |
| Nombre | Rol | Responsabilidades |
| Senior S4 | Full Stack | Define la estrategia de documentación y soporte.<br>Supervisa la calidad de los manuales, guías y procedimientos. |
| Junior S4/1 | Back-end<br>Front-end | Desarrolla herramientas internas para soporte y mantenimiento.<br>Automatiza tareas repetitivas del equipo de soporte. |
| Trainee S4/1 | Soporte | Atiende incidencias de primer nivel y documenta soluciones.<br>Colabora en la creación de guías para usuarios y técnicos. |

| | | |
|---|---|---|
| Trainee S4/2 | Soporte | Realiza pruebas de mantenimiento y verifica que las correcciones funcionen.<br>Mantiene actualizado el repositorio de documentación. |

B- Organigrama de la Organización del Proyecto EcoLogistics Web




*2. ENTORNO DE TRABAJO: INGENIERÍA ASISTIDA POR COMPUTADOR (CASE)*
*(Aplicación del documento: 1.1.8 - IS asistida por computador.pdf)*
*Ya no vale con usar "el bloc de notas". Como Ingenieros, debéis definir el ecosistema*
*de herramientas CASE (Computer Aided Software Engineering) que usará vuestro equipo para automatizar y controlar la calidad.*
*Tu Tarea:*
*Especifica en tu plan qué herramientas usaréis para cada nivel, clasificándolas según el documento 1.1.8:*
*Upper-CASE (Análisis y Diseño): ¿Qué software usaréis para dibujar los diagramas UML que veremos abajo? (Ej. StarUML, Lucidchart, herramientas de modelado de datos).*
*Lower-CASE (Codificación y Pruebas): Definir el IDE (VS Code, IntelliJ), el control de versiones (Git/GitHub) y herramientas de testing automatizado.*
*I-CASE (Integración): ¿Cómo vais a unir el diseño con el código?*
*Requisito: Debes justificar por qué la herramienta elegida ayuda a los alumnos de "Nivel Bajo" (ej. herramientas visuales que generan código automáticamente).*

A- Upper-CASE (Análisis y Diseño)

a) Herramienta seleccionada: Lucichart

Justificación:
Entre las dos herramientas analizadas, Lucichart destaca como la opción más adecuada para este proyecto, dado que al ser muy visual e intuitiva con multiples platillas que combina tanto la facilidad de uso como de aprendizaje para los trainees. Esta dinámica acelera el aprendizaje al reducir la complejidad y facilitar la explicación y mentoría de los seniors mientras se construyen los diagramas UML o cualquier otro tipo de diagrama necesario.Además se trata de una plataforma online que no requierre instalación, permitiendo al equipo no solamente trabajar en tiempo real sino también simultáneamente en el mismo diagrama, favoreciendo la colaboración desde el inicio de forma segura.

b) Herramientas Analizadas para el proyecto

| Herramienta | Característica | Por qué elegirla? | Beneficios para Trainees |
|---|---|---|---|
| Lucichart | Online | Es visual e intuitiva<br>Tiene plantillas para UML<br>Permite trabajar en tiempo real | Pueden aprender y preguntar en tiempo real de los Jrs. y Srs.<br>No hay necesidad de instalación |
| Draw.io | Online&Offline Gratuito | Es ligero y sin registro<br>Permite exportar a varios formatos<br>Se integra en Google drive & GitHub | Pueden practicar sin coste adicional<br>Pueden crear archivos en varios formatos<br>Pueden incorporar diagramas junto al código |
| **Recomendación:** Lucichart puesto al ver como trabajan los seniors, aprenden más, sienten mayor seguridad en su aprendizaje e impacta en positivo en su rendimiento. | | | |

B- Lower-CASE (Codificación y Pruebas)

a) Lower-CASE seleccionado: Entorno de Desarrollo Integrado - Visual Studio Code

Justificación:
Entorno de Desarrollo Integrado
Entre las herramientas elegimos Visual Studio Code porque es un entorno de desarrollo que facilita mucho el aprendizaje. Su interfaz es sencilla, muestra el código de forma clara y ofrece ayudas visuales (partes del código). El autocompletado y las extensiones son una guía constante que sugieren tanto funciones, como fallos y facilitan entender las estructuras de HTML, CSS, JavaScrpt; lo que para un trainee significa aprender al mismo tiempo que escribe código. Esta herramienta guía, acompaña y enseña con seguridad.

b) El Control de Versiones

Herramientas seleccionada: (Git/GitHub)

Justificación:
Elegimos Git + GitHub porque son herramientas fundamentales para aprender a trabajar como desarrollador desde el primer día.
Git por si sólo permite practicar con comandos básicos de control de versiones y ayuda a los trainees a entender cómo guardar los cambios y volver atrás si cualquier cosa falla y de esta forma poder ver la evolución del proyecto.

Por su parte GitHub, contribuye facilitando el trabajo en equipo al compartir el código, permite revisar las aportaciones hechas por los compañeros así como colaborar ordenadamente.
Para los trainees un gran beneficio ya que pueden experimentar sin miedo a perder código, y aprender en un entorno real y profesional.

c) Herramientas De Testing Automatizado seleccionadas:

Jest + Supertest & Cypress – Pruebas End-to-End (E2E) & k6 & OWASP ZAP

i. Jest + Supertest – Pruebas de API y unidad
Elegimos Jest y Supertest porque encajan con el backend en Node.js.
Jest es una herramienta ampliamente usada tanto frontend como backend; y Supertest permite comprobar con facilidad la respuesta de los endpoints de los APIs.
Los tests se realizan rapidamente y por ello el código puede ser validados antes de subirlo. Para los trainees el aprendizaje es fácil y les permite realizar pruebas simples.
Además es gratuito.

ii. Cypress – Pruebas End-to-End (E2E)
Elegimos Cypress porque permite probar la aplicación como si fuera un usuario real tanto abriendo un navigador, como con escritura y navegación. Esto nos permite simular y validar los flujos del ciudadado y también del conductor.
Cypress tiene amplios beneficios para los trainees porque muestra tanto videos como capturas de pantalla y permite entender lo que ha ocurrido. Su interfaz en tiempo real hace que escribir tests sea más visual y menos intimidante. Es gratuito y dispone de una gran comunidad. La herramienta detecta errores que las prueba de API no pueden ver y además la aplicación funciona de principio a fin con botones, formularios, mensajes, etc.

iii. k6 – Pruebas de carga y rendimiento
Elegimos k6 porque permite simular cientos o miles de usuarios a la vez, algo que es necesario dado que Ecologistics Web debe soportar “miles de ciudadanos conectados simultáneamente”.
Es una herramienta ligera, que puede ejecutarse en el pipeline sin ralentizar el sistema. Además, usa también JavaScript facilitando al equipo completo que pueda escribir scripts sin tener que aprender un nuevo lenguaje. Es una aplicación gratuita que permite definir umbrales de rendimiento. Esta herramienta garantiza la buena experiencia del ciudadano y de los empleados dado que asegura el funcionamiento de la plataforma Ecologistics al evitar las caidas y la lentitud del sistema causadas por la alta demanda.

iv. OWASP ZAP – Pruebas de seguridad
Añadimos a las anteriores herramientas OWASP ZAP con el fin de añadir desde el incio una herramienta de seguridad y protección de datos. Esta herramienta detecta automáticamente cualquier vulnerabilidad web. Su modo automático es ideal para trainees ya que permite un escaneo completo en pocos clics sin ser expertos en ciberseguridad. Además, se integra con facilidad con CI/CD analizando cada cambio en el código. Es una herramienta de código abierto que facilita cumplir con el Reglamento General de Protección de Datos (RGPD) evitando así que puedan filtrase datos sensibles de los ciudadanos que accedan a la plataforma y protegiendo la aplicación frente a cualquier primer ataque.

| Lower-Case (Codificación y pruebas) | | | |
|---|---|---|---|
| Entorno de Desarrollo Integrado (IDE) | | | |
| (IDE) | Característica | Por qué elegirla? | Beneficios para Trainees |
| VS Code | Gratuito<br>Open source | - Curva de aprendizaje baja-media. Interfaz | - La interfaz limpia no abruma a los trainees al empezar. |

| | | | |
|---|---|---|---|
| | Sin limitaciones | limpia e intuitiva con muchos tutoriales disponibles<br>- Peso ligero, buen rendimiento en cualquier equipo<br>- Configuración incial minima<br>- Amplias extensiones, autocompletado y detección de errores<br>-Depuración integrada muy buena<br>Estándar en la industria general<br>- Amplia comunidad con foros y videos de ayuda | - Los cambios se ven rapidamente en HTML/CSS en el navegador lo que es motivador y permite corregir rapidamente errores.<br>- Permite un apredizaje gradual desde lo básico e ir añadiendo extensiones según necesidades.<br>- La comunidad permite aprender con ayuda de videos y tutoriales |
| IntelliJ IDEA | Community Edition gratuita | - Curva de aprendizaje media-alta debido al autocompletado inteligente. Interfaz profesional pero con tutoriales integrados<br>- Peso moderado-pesado aunque herramientas potentes<br>- Configuración inicial requiere configuración pero ayuda asistencial<br>- Amplias extensiones pero enfocadas a Java/Kotlin<br>- Excelente depuración visual (profunda)<br>- Estándar en la industria de Java por profesionales | - La refactorización es segura<br>- Es la mejor opción si se trata de un proyecto en Java<br>- Su autocompletado inteligente y detección de errores en tiempo real permite un aprendizaje guiado<br>- El plugin Key Promoter X enseña progresivamente los atajos de teclado |
| Recomendación: Para el proyecto EcoLogistics propongo VS Code como IDE estándar para todo el equipo, por su versatilidad y facilidad de aprendizaje para los trainees. Además VS Code es una herarmienta que se puede utilizar tanto en back-end como en front-end. Por otro lado sería complementado con IntelliJ IDEA solo para seniors en tareas específicas. Esto garantiza la rápida incorporación de los trainees y potencia la técnica cuando se necesita. | | | |
| | | | |
| El Control Versiones - Git/Github | | | |
| El control Versiones | Por qué elegirla? | | Beneficios para Trainees |
| Git / GitHub | - Es el estandar de la industria de desarrolladores (Github)<br>- Pull Requests (PRs) con Revisión de Código de GitHbub permite sugerir mejoras y explicar conceptos<br>- GitHub Actions (CI/CD integrado) que permite automatizar pruebas y despliegues directamente desde el repositorio<br>- Soporte online y amplia comunidad con tutoriales | | - Aprendizaje por observación al ver y estudiar el código de otros compañeros (Srs.y Jrs.): revisión de código formativo permite que los seniors expliquen mejoras y compartan su conocimiento<br>- Entorno seguro dado que las ramas son seguras para experimentar sin romper el proyecto principal<br>- Los tests automáticos en GitHub Actions les dan feedback inmediato |

| | | | |
|---|---|---|---|
| | El historial es visible y permite ver la evolución del proyecto y colaborar conjuntamente<br>- Perfiles públicos que pueden mostrar la contribución de los portfolios | | (CI/CD)<br>-Herramientas visuales dado que la integración con VS Code permite ver más allá de los comandos:el aprendizaje gradual para los trainees primero la interfaz es gráfica y más adelante permite profundizar en más comandos |
| | | | |
| Herramientas De Testing Automatizado | | | |
| Testing | Tipo de Test | Por qué elegirla? | Beneficios para Trainees |
| Jest | Unitario (JS) | - Se utiliza en JavaScript/React (frontend & Back-end)<br>- Configuración cero en proyectos modernos<br>- Mensajes de error extremadamente claros<br>- Nivel muy fácil | - Tiene una configuración sencilla que permite que los trainees puedan escribir facilmente tests.<br>- Tiene mensajes de error claros para entender facilmente lo que falla y por qué.<br>- interfaz de usuario visual que permite identificar cambios inesperados<br>- Fácil visualización del código testeado |
| Mocha +Chai | Unitario (JS) | - Se utiliza en Node.js (back-end)<br>- Permite flexibilidad total para estructurar tests<br>- Tiene un nivel de aprendizaje medio | - Interfaz personalizable para fomentar la autonomía<br>- Personalización de pruebas según necesidades del trainee para aprender<br>- Tiene mensajes descriptivos de errores y soluciones<br>- Sintaxis y estilo legible<br>- Alta adaptabilidad |
| JUnit | Unitario (Java) | - Es el estándar de facto para Java<br>- Tiene amplia comunidad y documentación<br>- Tiene nivel medio de aprendizaje | - preconfigurado en IntelliJ y VS Code<br>- Múltiples aserciones<br>- Estructura clara |
| Pytest | Unitario (Python) | - Utilizado para Python<br>- Sintaxis sencilla y legible<br>- utilizado en ciencia de datos y startups<br>-Aprendizaje muy fácil | -Curva de aprendizaje mínima<br>- Muestra de errores muy descriptivos (colores, etc.)<br>- Reutilización decódigo de preparación de pruebas de forma limpia y comprensible<br>-Ejecución de pruebas selectiva |
| Cypress | E2E (Web) | - Se ejecuta directamente en el navegador<br>- Recarga en tiempo real al escribir los tests<br>- Depuración visual | -Los trainees pueden ver sus pruebas como se arealizan en tiempo real<br>- Depuración en tiempo real<br>- Grabación de las interacciones<br>- Mensajes de error claros |
| Selenium | E2E (Web) | - Es el más antiguo<br>- Soporta múltiples lenguajes (Java, Python, C#, etc.)<br>- Es el más flexible y personalizable | - Es flexible y personalizable, pero requiere compresión del código.<br>- Java o Python permite escribir pruebas<br>- Requiere preparación profesional (seniors la usan para integraciones |

| | | | |
|---|---|---|---|
| | | | complejas) |
| Postman | API | -Testea las APIs con interfaz gráfica<br>- Es visual e intuitivo<br>- Dispone de monitores programados<br>- Colecciones, entornos y tests automatizados | - Los trainees pueden probar APIs y cómo funcionan sin tener que escribir código (formularios)<br>- Fomenta la colaboración al compartir colecciones organizadas<br>- Tests visuales (con JavaScript) y resultados inmediatos<br>-automatización en el pipeline (integración CI/CD) |
| REST Assured | API | - Es el estándar en testing De APIs en proyectos de Java<br>- Se integra con JUnit<br>- Sintaxis fluida tipo Given-When-Then | - La structura (Sintaxis BDD) es muy fácil de entender<br>- Al estar basado en código, se integra perfectamente en el pipeline de CI/CD |
| K6 | Carga y Rendimiento | - Para Scripts en JavaScript<br>- Es ligero y pensado para CI/CD<br>- Incluye reportes gráficos integrados con Grafana | - Los trainees pueden escribir pruebas de carga con el mismo lenguaje que usan en el frontend<br>- Es muy ligero y fácil de ejecutar<br>- Los resultados se muestran en gráficos y tablas que facilitan la comprensión de métricas<br>- Integración en CI/CD |
| Apache JMeter | Carga y Rendimiento | - Es el más antiguo<br>-Tiene la interfaz gráfica para diseñar pruebas<br>- Soporta múltiples protocolos | -Interfaz gráfica intuitiva sin necesidad de programar inicialmente<br>- Multi-protocolo HTTP y FTP, JMS, etc.<br>- Tiene capacidad para pruebas distribuidas |
| Locus | | -Para pruebas en Python<br>-Tiene interfaz web en tiempo real | - Perfecto si el trainee tiene soltura con Python<br>-Interfaz web en tiempo real<br>-permite escribir escenarios de usuario como código<br>-Open source activo |
| Recomentación general:<br>La mejor opción para pruebas Unitario (JS) es Jest debido a su configuración cero, a que los mensajes de error son claros, y la interfaz es sencilla para los trainees.<br>Cypress Interfaz gráfica, depuración visual, grabación de interacciones, recomendaciones IA<br>Postman Interfaz gráfica, sin código inicial, ideal para entender APIs<br>K6 usa JavaScript (mismo que frontend); JMeter tiene GUI visual<br>Postman es imprescindible para que todos (trainees incluidos) puedan explorar y probar la API. Luego, los tests automáticos se escriben con REST Assured (Java) o Supertest (Node.js)<br>K6 es la opción moderna y la que mejor se alinea con las tecnologías web actuales. Además, al usar JavaScript, los trainees no aprenden un lenguaje nuevo. | | | |

C- I-CASE (Integración)

a) Herramienta seleccionada: GitHub

Justificación:
Elegimos GitHub Actions porque permite que las pruebas se ejecuten automáticamente cada vez que alguien sube código al repositorío, lo que garantiza que la calidad sea revisada de forma contínua sin la implicación de los trainees. Al recibir feedback

inmediato pueden ver al instante si algo falla, entender qué ha pasado y corregirlo al instante. Todo esto reduce errores, ayuda a adoptar buenas prácticas y aumenta su curva de aprendizaje. GitHub Actions es una herramienta que acompaña y y facilita el aprendiza de los trainees.

| I-Case (Integración) | | | |
|---|---|---|---|
| Herramienta | Función | Por qué elegirla? | Beneficios para Trainees |
| GitHub Actions | Automatización CI/CD | - Integración nativa con GitHub<br>-Marketplace con miles de acciones reutilizables<br>- Multiplataforma para tests<br>- Curva de aprendizaje baja para trainees por su integración visual<br>- Gratuito para repositorios privados | -Feedback inmediato y automático:ejecuta tests sin que el trainee tenga que acordarse<br>- Visibilidad del proceso por el trainee<br>- Autonomía por parte del trainee |
| SonarCloud | Análisis continuo de calidad y seguridad del código | - Integración nativa con GitHub<br>- definen umbrales objetivos de calidad<br>- Es el estándar en la industria | - Ayuda 24h: detecta problemas y explica cómo corregirlos<br>- Los trainees reciben comentarios directamente en el PR y pueden corregirlos |
| Swagger / OpenAPI | Documentación interactiva y automatizada de la API | -Estándar abierto para describir APIs REST<br>-Generación automática desde anotaciones en el código<br>-Interfaz gráfica para probar endpoints directamente desde el navegador<br>-Integrable en GitHub Actions | - Comprensión visual de la API por los trainees<br>- Los trainees pueden probar la API sin herramientas externas<br>- Los trainees pueden trabajar en paralelo frontend-backend<br>- la documentación no queda obsoleta ya que se genera desde el código |
| Jenkins | Automatización CI/CD | -Requiere administrar un servidor propio<br>-Configuración compleja | - Comprenden el concepto de pipeline<br>- Pierden el miedo a la automatización |
| CircleCI | Automatización CI/CD | -Integración nativa con GitHub aunque menos que GitHubActions | - Comprenden la importancia de los tests automáticos<br>- Experimentan con pipelines ligeros y rápidos |
| CodeClimate | Análisis de calidad de código | - Funcionalidades gratuitas limitadas<br>-No ofrece un Quality Gate<br>-Integración menos profunda que GitHub Actions | -Desarrollan criterio técnico<br>- Comprenden por qué un código limpio importa<br>- Reciben feedback objetivo<br>- Aprenden a leer métricas de calidad |
| Recomendación: La mejor opción de forma general para los trainees para que puedan trabajar de manera autónoma con una mínima curva de aprendizaje es el GitHub Actions. Esta herramienta dispone de un feedback visual inmediato así como la integración con herramientas que en principio ya utilizan como VS Code y GitHub (ver Lower-Case).<br>Además, esta herramienta es suficientemente potente para pipelines complejos y dispone de escalabilidad probada que es útil para los seniors. | | | |

## *3. ARQUITECTURA DEL SOFTWARE: EL ENFOQUE HÍBRIDO*
***(Aplicación de los documentos: 1.1.5 Estructurada y 1.1.6 Orientada a Objetos)***
***Aquí está el núcleo de la ingeniería. EcoLogistics es complejo, por lo que usaremos una arquitectura mixta.***

Para el sistema Ecologistics, y dado que se trata de una solución con cierta complejidad, hemos optado por una arquitectura hibrida que combina dos enfoques:

- Por una lado, la Programación orientada a Objetos (OOP), que nos permite representar de forma natural las entidades del mundo real (conductores, rutas o contenedores) y las relaciones entre ellas.

- Por otro, la ingeniería Estructurada, que resulta muy útil para diseñar aquellos algorithmos críticos que necesitan una lógica clara, ordenada y altamente eficiente.

De esta manera aprovechamos lo mejor de ambos mundos. Los modelos bien definidos y fáciles de antener, junto con los procesos internos optimizados para las partes más delicadas del sistema.

***El Corazón del Sistema - Orientación a Objetos (OOP)***
***(Ref: 1.1.6 - IS orientada a objetos.pdf)***
***El sistema debe modelar la realidad. Debes diseñar el Diagrama de Clases conceptual.***
***Tu Tarea: Define las clases principales, sus atributos y métodos, aplicando Encapsulamiento y Herencia.***
***• Ejemplo a desarrollar:***
***Clase Padre: Usuario (id, nombre, login()).***
***Clases Hijas: Ciudadano (dirección, reportar_incidencia()) y Conductor (licencia, ver_ruta()).***
***Objeto Físico: Contenedor (tipo, capacidad, estado_llenado).***
***Pregunta clave: ¿Cómo aplicas el Polimorfismo en el método recogerBasura() dependiendo si el camión es de vidrio o de orgánico?***

A- Diseño Orientado a Objetos (OOP)
Para el diseño del sistema hemos creado un modelo basado en OOP, donde vamos a definir las principales clases del dominio.

a) Clases de Dominio

i. Usuario

Usuario como clase base y sus derivadas Ciudadano, Contenedor y Administrador.

ii. Contenedor y Ruta

Contenedor y Ruta como clases que representan elementos clave del sistema.
Aplicamos los principios de la orientación a objetos, como el encapasulamiento (atributos privados y métodos públicos) y la herencia, para estructurar las relaciones entre las clases y mantener un diseño claro y fácil de mantener.

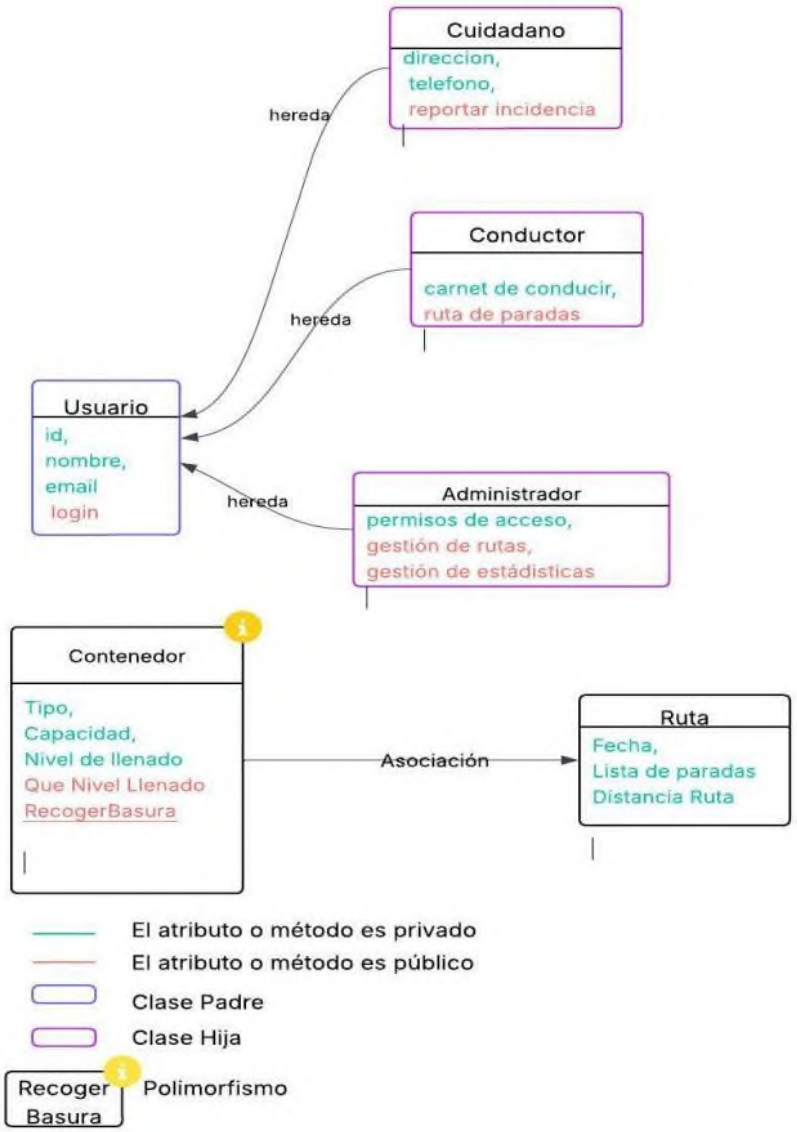
iii. Polimorfismo

En este caso, el polimorfismo lo vamos ver en para el método recogerBasura de la clase Contenedor que cambia su comportamiento según el tipo de camión que realiza la recogida. Dependiendo del vehículo que recoge (camión) su comportamiento será diferente aunque la acción recogerBasura sea la misma.

- Si el camión es de de recogida de vidrío su respuesta es: “Despositar en la planta de vidrío para el vidrío sea fundido”

- Si el camción es de recogida de orgánico su respuesta es: “Despositar en la planta de compostaje”.

b) Diagrama de Clases del Diseño Orientado a Objetos (OOP)




*Los Procesos Críticos - Ingeniería Estructurada*
*(Ref: 1.1.5 . IS estructurada.pdf)*
*Aunque usemos objetos, hay algoritmos internos que requieren un pensamiento estructurado (Top-Down) para ser eficientes, especialmente para los alumnos de nivel*
*medio que se lían con la OOP.*
*Tu Tarea: Diseña el Diagrama de Flujo (Pseudocódigo estructurado) para el algoritmo de "Asignación de Rutas".*
*Lógica:*
*INICIO*
*Leer lista de contenedores.*

*MIENTRAS (camión no lleno) HACER:*
*SI (contenedor > 70% lleno) ENTONCES añadir a ruta.*
*SINO ignorar.*
*FIN.• Debes usar las 3 estructuras básicas: Secuencia, Selección (If) e Iteración (While/For)*
*explicadas en el documento 1.1.5.*

B. Ingeniería Estructurada - Los Procesos Críticos

a) Explicación del diagrama de Flujo para el Algoritmo "Asignación de Rutas"

Paso 1: INICIO de la ruta
En este paso se crea la ruta

Paso 2: Leer lista de contenedores
En este paso se reunen en una lista todos los contenedores de la cuidad o municipio cuyas rutas se van a crear

Paso 3: Crear lista vacia de paradas
Se crea una lista vacía de paradas para ir guardando aquellos contenedores que se vayan recogiendo. Esta lista se va construyendo poco a poco a base de de aquellos contenedores que van superando el nivel de llenado del 70%.

Paso 4: Leer lista de camiones disponibles
Se crea una lista aquellos camiones que están vacios o disponen de espacio para ser rellenados con el contenido de los contenedores.

Paso 5: SI (contenedor< 70% lleno) ENTONCES ir a paso 9
El sistema pregunta si el camión de la lista está lleno en relación al nivel de llenado (superior al 70% de llenado)

Paso 6: MIENTRAS Camión no lleno HACER: ir siguiente contenedor
Si el camión no está lleno se selecciona el siguiente contenedor de la lista

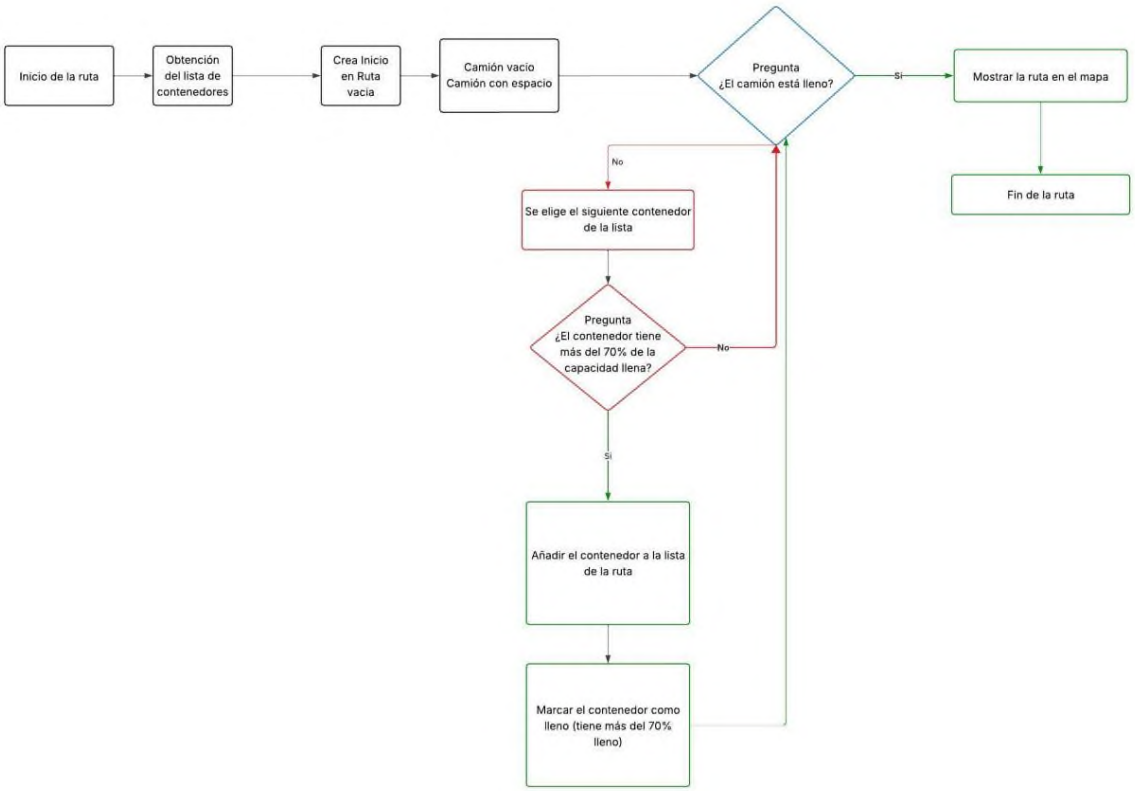
Paso 7: MIENTRAS Camión no lleno HACER: ir siguiente contenedor con nivel >70%
El sistema pregunta si el contenedor tiene un nivel de llenado mayor al 70%

Paso 8 : MIENTRAS Camión no lleno HACER: ir guardando contenedores con nivel >70%
Si el nivel de llenado del contenedor es superior al 70% entonces es añadido a la lista de la ruta de recogida y el camión se marca el camión que va encargarse de recoger el contenedor.
Y vuelve al paso 5 para seguir la revisión nuevamente desde este paso otra vez.

Paso 9: SINO Ignorar
Si todos los camiones son evaluados correctamente y el camión está lleno se pasa directamente al paso 9. En este paso se crea el mapa con la ruta del día para el conductor del camión.
En el paso 5 que pregunta si exiten contenedores evaluados es cuando se define la lista de ruta final.

Paso 10 : FIN
Finalización de la creación de la lista de camiones y contenedores con el nivel superior al 70% de llenado que crean de la ruta del camión y que será entregada al condutor para su trabajo por parte del sistema en lista y en un mapa.

b) Diagrama de Flujo para el Algoritmo “Asignación de Rutas”




## *4. IMPLEMENTACIÓN: INGENIERÍA WEB*
*(Aplicación del documento: 1.1.7 - Ingeniería Web.pdf)*
*Finalmente, todo esto debe funcionar en un navegador. Basándote en el documento*
*de Ingeniería Web, debes definir las características no funcionales y la arquitectura de despliegue.*
*Tu Tarea:*

### *A. Separación Frontend/Backend*
*Explica a tus alumnos novatos por qué no pueden mezclar el código de la base de datos con el HTML (Principio de separación de capas).*

a) ¿Por qué es mala idea mezclar HTML con código de base de datos?

Para explicar el principio de separación de capas vamos a imaginar que estamos preparando un bocadillo de tal forma que:
El HTML es el pan que es lo que se ve por fuera. La base de datos es el rellenos del bocadillo (jamón, queso, etc.) que es lo que está dentro. El backend es la mano de la persona que está preparando el bocadillo. De esta forma, no se puede mezclar el orden de las diferentes capas ya que no podríamos comernos el bocadillo. No podríamos meter la mano dentro del pan al comer el bocadillo o poner el jamón por fuera del pan si lo que queremos es un bocadillo. De la misma manera, no tiene sentido mezclar el código de la base de datos con el HTML ya que no tiene sentido o no funciona, y los errores que se pueden producir son los siguientes:

i. Es muy dificil de mantener si las consultas que se producen a la base de datos están metidas dentro del HTML ya que cualquier cambio obliga a revisar multitud de archivos.

ii. Se produce un riesgo de seguridad si el código accede a la base de datos que contiene las contraseñas y las consultas. Mezclar la base de datos con HTML puede hacer llegar información privada del navegador al usuario ya que es una entrada al sistema.

iii. Cuando todo está mezclado cualquier pequeño fallo puede dar problemas y complicar la situación para encontrar los errores sobretodo si se trata de desarrolladores principiantes a los que les cueste encontrar un pequeño fallo.

iv. Si la base de datos y el HTML está juntos o revueltos se corre el riesgo de romper todo o aquella parte en la que el equipo o un squad esté trabajando. Lo que también evita la colaboración de diferente tipo de perfiles al mismo tiempo.

v. Si la base de datos está junto al HTML no se puede reutilizar el código en la app móvil y al mismo tiempo en la web y en una API ya que se multiplican los errores.

Aplicación del principio de separación de capas en Ecologistics Web

- Frontend (HTML, CSS, JS del navegador) - lenguaje: HTML, CSS, JavaScript
Aqui se desarrollan los botones, mapas y formularios.
Se envía peticiones al servidor pero no toca la base de datos
La ventaja es que el diseñador puede cambiar el HTML sin romper nada.

- Backend (Node.js + Express) - lenguaje: JavaScript
En esta capa es donde se sitúa la lógica. Además, se reciben las peticiones, se validan los datos y se consulta la base de datos. Las respuestas se devuelven en JSON. El backend puede mejorar las consultas sin afectar al diseño del frontend.

- Base de datos (PostgreSQL) - lenguaje: SQL
En esta capa solo se almacena y se devueñve la información. En ningún momento se tiene contacto con HTML. Los senior se encargan de configurar la base de datos.

- Comunicación (API Rest) - lenguaje: JSON
El servidor envía la comunicación al navegador en formato JSON.

-> Con esta estructura en capas el equipo puede trabajar en paralelo sin bloquearse y sin romper el código además de poder realizar pair Programming.

***B. La "Nube" y la Concurrencia***
***EcoLogistics tendrá miles de ciudadanos conectados a la vez.***
***• ¿Cómo garantizas la Escalabilidad? (Ref. pág. 13 del doc 1.1.7).***
***• ¿Qué medidas de Seguridad implementarás? (Menciona al menos la protección de datos en el servidor y validación de formularios).***

a) ¿Cómo garantizamos la escalabilidad del sistema EcoLogistics?
Cuando cientos o miles de ciudadanos entren al mismo tiempo el sistema debe siempre estar funcionando sin caidas, ni lentamente. Ya que cuando un ciudadano quiera reportar una incidencia o consultar el estado de su incidencia necesitamos que su experiencia sea de éxito. Para lograr esta experiencia utilizaremos dos mecanismo que trabajan al unisono el autoescalado y el balanceo de la carga.

i. Autoescalado
El autoescalado facilita que la infraestructura crezca o se reduzca automáticamente según la cantidad de usuarios que están conectados al mismo tiempo.
Durante un pico de usuarios conectados la nube crea más servidores para repartir el trabajo y cuando la actividad baja se eliminan los servidores creados ahorrando en recursos.

Con este mecanismo se evitan los cuellos de botella y se mantiene el fujo.

ii. El Balanceo de carga
El balanceo de carga o Load Balancing permite que cada petición de los ciudadanos sea enviada al servidor que esté más liberado en ese momento, evitando que un determinado servidor se sature mientras otros estén vacios. Esto corresponde a un reparto inteligente donde la aplicación responde rapidamente en momentos de máxima demanda y creando una situación de equilibrio.

b) ¿Qué medidas de Seguridad implementar?
Para proteger la información de los ciudadanos y asegurar que la aplicación funcione de forma segura, incorporamos varias capas de protección que van a actuar conjuntamente.

i. Contraseñas protegidas (hashing)
Las contraseñas nunca deben guardarse en la base de datos tal cual. Lo que vamos a hacer es transformarlas mediante un algoritmo de hash (bcrypt) que genera una versión cifrada que no puede revertirse. De esta forma, incluso en el caso de que alguien consiguiera acceder a la base de datos así no podría obtener la contraseña original.

ii. Validación de formularios en 2 fases

* Validación en el navegador (cliente)
Antes de enviar el formulario al servidor el navegador comprueba que los campos obligatorios esté correctamente rellenos y tengan el formato adecuado, como por ejemplo el número de teléfono, el email, la contraseña, etc. De esta forma además de ayudar al usuario a completar el formulario, se evita enviar al servidor peticiones erroneas e innecesarias.

* Validación en el servidor (backend)
El servidor realiza la comprobación fundamental ya que bloquea los intentos de código malicioso (SQL, XSS, scripts peligrosos en la web). Además, limpia las entradas para eliminar cualquier tipo de caracter que pueda causar errores o problemas.

iii. Comunicación cifrada (HTTPS)
La información que existe entre el navegador y los servidores se produce mediante HTTPS, es decir cifrada. Este tipo de comunicación al ser completamente cifrada evita que tanto contraseñas, datos personales como todo tipo de información sensible pueda ser interceptada por terceros.

***C. Tecnologías Web***
***Define el stack concreto.***
***◦ Frontend: HTML5/CSS3 + Framework JS (React/Vue).***
***◦ Backend: API REST (NodeJS/Python/Java).***
***◦ Justificación: ¿Por qué estas tecnologías favorecen el mantenimiento (Maintenance) mencionado en el documento?***

a) Tecnologías Web - Definición del Stack y Justificación

i. Frontend HTML5-CSS3 & JavaScript (React/Vue)
Para la parte de Frontend del sistema vamso a utilizar HTML5 y CSS3 junto con un framework moderno React. Estas frameworks permiten construir interfaces dinámicas, modulares y fáciles de mantener gracias a su sistema reutilizable.

ii. Backend API REST (Node.js & Java)
La lógica del servidor que vamos a utilizar se implimenta mediante API REST desarrollada con Node.js y Java. Con este enfoque podemos seperar completamente la interfaz del procesamiento interno facilitando de esta manera la escalabilidad y el mantenimiento.

**Justificación**
¿Por qué este stack mejora el mantenimiento?
La idea es que el sistema sea fácil de extender, depurar y actualizar en el tiempo.

iii. Separación entre capas
Frontend & backend funcionan de manera independiente
El Frontend gestiona unicamente la interfaz.
El backend gestiona la lógica, las validaciones y el acceso a los datos.
De esta forma se puede modificar la interfaz sin actuar en el servidor ni actualizar la API.
La separación por capas reduce los errores y facilita el trabajo en equipo.

iv. Frontend con componentes reutilizables
React permite crear componentes independientes (botones, formularios, etc). Cuando un componente se actualiza, no hay que reescribir el código ya que la actualización es automática y evita la duplicidades.

v. APIs REST
La utilización de APIs REST facilita una arquitectura flexble y estándar así como un mantenimiento a largo plazo. Facilita añadir una app móvil, e integrar servicios externos.

vi. Node.js & Java& Python
Facilitan un código limpio y organizado que es fácil de leer, entender, modificar incluso para trainees.
Además son herramientas altamente documentadas y maduras que facilitan la resolución de problemas, y reducen el tiempo de trabajo. Tienen amplias facilidades como librerías estables, grandes comunidades, amplias extensiones, ayudan a depurar, y tienen frecuentes actualizaciones.

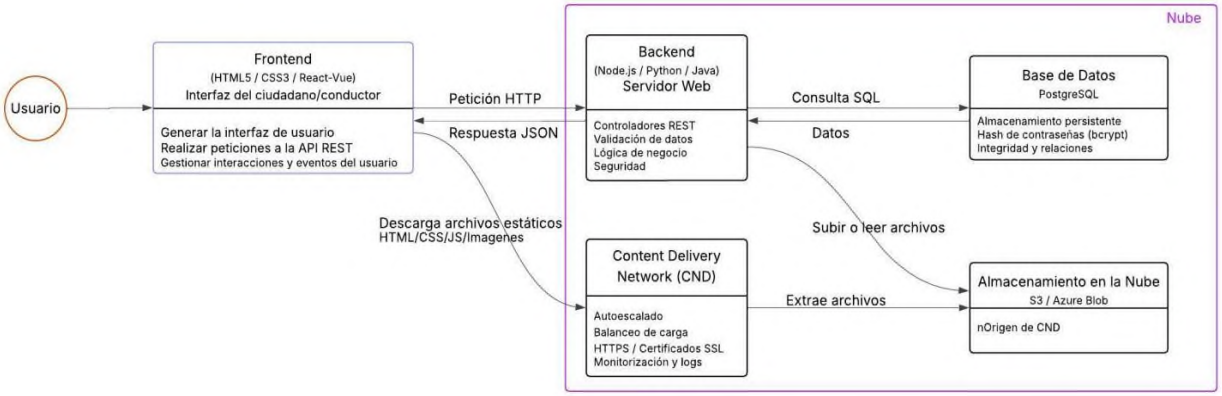
En resumen, el uso de React en el Frontend y Node.js en el backend unifica en JavaScript el leguaje de programación para todas las capas del sistema. Esto simplifica el desarrollo: el equipo no necesit cambiar de metalidad ni de sintaxis al pasar del cliente al servidor, lo que acelera la resolución de errores y reduce la curva de aprendizaje.
La arquitectura basada en componentes de React facilita realizar cambios localizados sin afectar al resto de la interfaz. Cada pieza es independiente y reutilizable por lo que las modificaciones son más seguras y rápidas.

En el servidor, una API REST desacoplia completamente el frontend del backend. De esta forma ambas partes pueden evolucionar por separado y se puede actualizar la interfaz sin tocar la lógica del servidor o mejorar la API sin actuar en la experia del usuario.
En el caso de PostgreSQL aporta estabilidad a la base de datos con el fin de que pueda crecer con el tiempo.

En general, este stack tecnológico hace que el coste de mantenimiento de Ecologistics Web sea mucho menor que si se utilizasen tecnologías heterogéneas o herramientas de pago más dificiles de aprender y de mayor coste.

EcoLogistics Web sea significativamente menor que si se usaran tecnologías dispares o propietarias.

b) Diagrama de Arquitectura Web (navegador – servidor – BBDD)




*Estructura del documento que debéis entregarme:*
*• Organigrama del Equipo: (Cómo has mezclado los 21 alumnos).*
*• Selección CASE: (Tu caja de herramientas).*
*• Modelo de Datos (OOP): Diagrama de clases preliminar.*
*• Algoritmo Crítico (Estructurado): Pseudocódigo del sistema de rutas.*
*• Arquitectura Web: Diagrama de cómo se conectan el navegador, el servidor y la BBDD.*

Curso:  Metodología y gestión de proyectos de Ingeniería Software (IS)

Proyecto: EcoLogistics Web (Sistema Inteligente de Gestión de Residuos)

Responsable Académico: José Luis Martínez Vila

Responsable de Ejecución: Angelina Caballero (Rol:Estudiante/Project Manager)

Semana:  2 Año: 2026

# TABLA DE CONTENIDOS

*ENTREGABLE 1: LA INGENIERÍA DE REQUISITOS (ERS)* ..... 3
*1. Requisitos Funcionales (El Qué)* ..... 3
A- Catalogo de Requisitos Funcionales (RF) ..... 3
a) Tabla Catalogo de Requisitos Funcionales (RF) (organizados por actor): ..... 3
*2. Requisitos No Funcionales (Restricciones)* ..... 4
A- Catalogo de Requisitos No Funcionales (RNF) ..... 4
a) Tabla de Catálogo de Requisitos No Funcionales (RNF) ..... 4
*3. Matriz de Trazabilidad (Ref. 1.2.3)* ..... 5
A - Matriz de Trazabilidad ..... 5
*ENTREGABLE 2: ANÁLISIS DE PROCESOS Y DATOS* ..... 5
*1. Diagrama de Flujo de Datos (Ref. 1.2.4)* ..... 5
A- Diagrama de Flujo de Datos (DFD) ..... 6
*2. Diccionario de Datos Conceptual (Ref. 1.2.5)* ..... 6
A - Principales Entidades ..... 7
B - Relaciones y cardinalidades ..... 8
*ENTREGABLE 3: ANÁLISIS ORIENTADO A OBJETOS CON UML* ..... 8
*1. Diagrama de Casos de Uso (Ref. 1.2.7)* ..... 8
A- Tabla del Diagrama de Casos de Uso ..... 8
B- Diagrama de Casos de Uso ..... 9
*2. Diagrama de Clases del Dominio (Ref. 1.2.6)* ..... 9
A- Tabla de Clases según los Atributos ..... 9
B - Herencia ..... 10
C - Encapsulamiento ..... 11
a) Tabla de Encapsulamiento para EcoLogistics Web ..... 11
D- Relaciones y Cardinalidad ..... 11
E - Diagrama de Clases del Dominio ..... 12
*3. Diagrama de Secuencia (Ref. 1.2.7)* ..... 12
A- Caso del Ciudadano que reporta un contenedor lleno. ..... 13
a) Secuencia de tiempos ..... 13
b) Diagrama de Secuencia ..... 14
*INSTRUCCIONES PARA EL JEFE DE PROYECTO (ALUMNO)* ..... 14
1. Organización del Equipo por Perfiles ..... 14

**Semana 2 – Especificación de Requisitos Software (ERS)**

*ENTREGABLE 1: LA INGENIERÍA DE REQUISITOS (ERS)*
*1. Requisitos Funcionales (El Qué)*

*1. Catálogo de Requisitos (Ref. 1.2.2)*
*Debes listar los requisitos funcionales (RF) y no funcionales (RNF), asignándoles un ID único.*
*Ejemplo: RF-001: El sistema debe permitir al ciudadano geolocalizar un contenedor mediante GPS.*
*Ejemplo: RF-002: El sistema calculará la ruta óptima basándose en el nivel de llenado > 70%.*

A- Catalogo de Requisitos Funcionales (RF)

a) Tabla Catalogo de Requisitos Funcionales (RF) (organizados por actor):

| ID | Requisito | Descripción |
|---|---|---|
| A. Requisitos del Actor Usuario/Ciudadano | | |
| RF-001 | Registro de Usuario | El sistema debe permitir que un Usuario se registre proporcionando nombre, email y contraseña |
| RF-002 | Inicio de sesión | El Usuario debe poder iniciar sesión con sus credenciales (email y contraseña) |
| RF-003 | Geolocalización de contenedores | El sistema debe mostrar en un mapa interactivo los contenedores cercanos a la ubicación del Usuario. |
| RF-004 | Consultar el estado de un contenedor | El Usuario podrá ver el nivel de llenado actual de un contenedor específico. |
| RF-005 | Reporte de incidencias | El Usuario podrá reportar una incidencia seleccionando un contenedor e indicar el tipo de problema del contenedor |
| RF-006 | Historico de incidencias | El Usuario podrá consultar el listado de incidencias que ha reportado, con su estado actual (pendiente, en proceso, resuelta) |
| B. Requisitos del Actor Conductor | | |
| RF-007 | Registro del conductor | El sistema debe permitir que un Conductor se registre proporcionando nombre, matricula de vehículo |
| RF-008 | Inicio de sesión | El Conductor debe poder iniciar sesión con sus credenciales |
| RF-009 | Visualización de la ruta asignada | El conductor podrá ver la ruta del día con el listado ordenado de contenedores a visitar y su ubicación en el mapa |
| RF-010 | Marcar el contenedor como recogido | El conductor podrá indicar que ha vaciado un contenedor actualizando su nivel de llenado a 0% |
| RF-011 | Reportar incidencia en ruta | El conductor podrá reportar incidencias durante la ruta (contenedor dañado, imposibilidad de acceso, etc.) |
| RF-012 | Detalles del contenedor durante la ruta | Al seleccionar un contenedor de la ruta, el conductor podrá los detalles del contenedor (la dirección, el tipo de contenedor, el nivel de llenado) |
| RF-013 | Recibir notificaciones de aviso y detalles del contenedor | El conductor recibirá notificaciones cuando un contenedor supere el 75% de llenado |
| C. Requisitos del Actor Administrador | | |
| RF-014 | Inicio de sesión | El Administrador debe poder iniciar sesión con sus credenciales privilegiadas |
| RF-015 | Gestión de conductores | El administrador podrá dar de alta, modificar y dar de baja conductores en el sistema |

| | | |
|---|---|---|
| RF-016 | Asignación de rutas | El administrador podrá asignar a los conductores las rutas generadas automáticamente |
| RF-017 | Gestión de contenedores | El administrador podrá dar de alta, modificar y dar de baja los contenedores en el sistema |
| RF-018 | Consultar estado de contenedores | El administrador podrá consultar el estado y los detalles de todos los contenedores (nivel de llenado, última recogida, etc.) |
| RF-019 | Generación de informes | El administrador podrá generar informes sobre eficiencia de rutas,de contenedores, gestión de incidencias |
| D. Requisitos del Sistema | | |
| RF-020 | | |
| RF-021 | | |

*2. Requisitos No Funcionales (Restricciones)*
*Ejemplo: RNF-001 (Seguridad): Las contraseñas deben estar encriptadas (Ref. RGPD).*
*Ejemplo: RNF-002 (Rendimiento): El mapa de contenedores debe cargar en menos de 2 segundos.*

A- Catalogo de Requisitos No Funcionales (RNF)

a) Tabla de Catálogo de Requisitos No Funcionales (RNF)

| ID | Requisito | Categoria | Descripción |
|---|---|---|---|
| Requisitos por Usuario/Ciudadano | | | |
| RF-001 | Compatibilidad de navegadores | Portabilidad | La aplicación debe funcionar correctamente en las últimas versiones de los principales buscadores |
| RF-002 | Tiempo de carga del mapa | Rendimiento | El mapa con los contenedores debe cargar en menos de 2 segundos |
| RF-003 | Accesibilidad | Usabilidad | La interfaz web debe cumplir con las directrices WCAG |
| Requisitos por Conductor | | | |
| RF-004 | Tiempo de actividad | Disponibilidad | El sistema tendrá una disponibilidad del 99.5% en horario de servicio |
| RF-005 | Usuarios simultáneos | Concurrencia | El sistema debe soportar una capacidad de 10.000 usuarios simultáneos con respuesta inferior a 3 seg |
| Requisitos por Ayuntamiento | | | |
| RF-006 | Tiempo de actividad | Disponibilidad | El sistema tendrá una disponibilidad del 99.5% en horario de servicio |
| RF-007 | Encriptación de contraseñas | Seguridad | Todas las contraseñas de usuarios se almacenarán en la base de datos utilizando encriptación |
| RF-008 | Escalado horizontal | Escalabilidad | La arquitectura debe permitir añadir nuevas instancias del backend sin modificar el código |
| Requisitos del Equipo de Seguridad | | | |
| RF-009 | Encriptación de contraseñas | Seguridad | Todas las contraseñas de usuarios se almacenarán en la base de datos utilizando encriptación |
| RF-010 | Calidad del código | Mantenibilidad | El código fuente deberá mantener una cobertura de pruebas superior al 80% y pasar los tests de calidad |

*3. Matriz de Trazabilidad (Ref. 1.2.3)*
*Debéis crear una tabla que vincule el origen del requisito con su futuro desarrollo. .*
*Esto es vital para evitar el Scope Creep (alcance no controlado).*
*Columnas obligatorias: ID Requisito | Descripción | Fuente (quién lo pidió) | Prioridad (MoSCoW) | Estado*

Establecemos la siguiente definición de los estados, alineados con las fases de desarrollo:

| Estado | Definición |
|---|---|
| Pendiente | El requisito está identificado pero aún no se ha comenzado a trabajar |
| En análisis | Se está detallando el requisito y validando con el cliente.<br>Tambié se están definiendo los criterios de aceptación |
| En diseño | Se está diseñando la solución (diagramas, arquitectura, interfaz) antes de codificar |
| En desarrollo | El equipo está escribiendo el código y las pruebas correspondientes |
| En pruebas | El código está implementado y se están ejecutando pruebas (unitarias, integración, y aceptación) |
| Validado | El requisito ha sido probado y cumple los criterios de aceptación.<br>Está listo para ser entregado |
| Cerrado | El requisito ha sido entregado y aceptado formalmente |

A - Matriz de Trazabilidad

| ID | Descripción | Fuente (quién lo pidió) | Prioridad ((MoSCoW) | Estado |
|---|---|---|---|---|
| Solicitud por Usuario/Ciudadano | | | | |
| RF-001 | Autenticación y registro | Usuario/Ciudadano | Must have | Pendiente |
| RF-002 | Geolocalización de contenedores | Usuario/Ciudadano | Must have | Pendiente |
| RF-003 | Reporte de incidencias | Usuario/Ciudadano | Must have | En análisis |
| RF-004 | Historial de incidencias | Usuario/Ciudadano | Could have | En análisis |
| Solicitud por Conductor | | | | |
| RF-005 | Autenticación y registro | Conductor | Must have | Pendiente |
| RF-006 | Visualización de rutas | Conductor | Must have | Pendiente |
| RF-007 | Notificaciones automáticas | Conductor | Should have | En análisis |
| Solicitud por Ayuntamiento | | | | |
| RF-008 | Gestión de rutas óptimas | Ayuntamiento | Must have | En diseño |
| RF-009 | Gestión de contenedores | Ayuntamiento | Must have | Pendiente |
| RF-010 | Asignación de conductores | Ayuntamiento | Should have | Pendiente |
| RF-011 | Gestión de informes | Ayuntamiento | Could have | Pendiente |

*ENTREGABLE 2: ANÁLISIS DE PROCESOS Y DATOS*
*Antes de hablar de clases y objetos, debemos entender los flujos de información y la estructura de los datos brutos. (Referencias: 1.2.4 Funciones/Procesos y 1.2.5 Datos)*

*1. Diagrama de Flujo de Datos (Ref. 1.2.4)*
*Selecciona el proceso más complejo de EcoLogistics (ej: "Gestión de una incidencia de contenedor roto") y detalla:*
- *Entrada: ¿Qué datos entran? (Foto, ID contenedor, hora).*
- *Transformación: ¿Qué reglas de negocio aplican? (Verificar usuario, asignar*

*prioridad alta, notificar a camión de mantenimiento).*
*● Salida: ¿Qué se genera? (Orden de trabajo, notificación push al ciudadano).*

A- Diagrama de Flujo de Datos (DFD)

Procesos de EcoLogistics - Asignación inteligente de las rutas

✓ Entrada
El administrador introduce en el sistema los parámetros de optimización: nivel de llenado superior al 75%, horarios de recogida, etc. El proceso 1 (Filtrar los contenedores prioritarios) consulta la lista de Contenedores para saber cuales son aquellos cuyo nivel de llenado supere el 75% y generar una lista de contenedores prioritarios.

✓ Transformación
En el proceso 2 (Optimizar la ruta) simultáneamente se recibe la información de la lista de contenedores prioritarios del Mapa (distancia y tiempo real) y la lista de la capacidad de camiones. Con esta información se produce la mejor ruta y la lista de los contenedores que cada camión puede seguir.
Una vez optimizada la ruta se procede al proceso 3 (Asignar conductor y vehículo), que consulta la lista de Conductores disponibles para encontrar un conductor disponible para emparejar la ruta optima con un camión determinado (lista definitiva).

✓ Salida
En el proceso 4 (Guardar ruta y notificar ruta) se guarda la ruta definitiva en la lista de Rutas asignadas y envía una notificación al conductor con la ruta asignada para que sepa la ruta que le corresponde.

| | Descripción |
|---|---|
| Entrada<br>(¿Qué datos entran?) | -Parámetros de optimización de la mejor ruta<br>(nivel de llenado>75%, horario de recogida)<br>- Lista de contenedores<br>(ID, ubicación, nivel de llenado actual)<br>-Mapa obtenido de un servicio externo (ejemplo:Google Maps)<br>(distancia de contenedores, tiempos real de desplazamiento)<br>- Lista de la capacidad los camiones<br>(capacidad disponible del camión)<br>-Lista de conductores disponibles<br>(ID,horario o turno)<br>-lista de rutas asignadas<br>(ruta optima, camión asignado, conductor asignado) |
| Transformación<br>(¿Qué reglas de negocio aplican?) | - Proceso 1: Filtrar los contenedores prioritarios<br>- Proceso 2: Optimizar la ruta<br>- Proceso 3: Asignar conductor y vehículo<br>- Proceso 4:Guardar ruta definitiva y notificar ruta |
| Salida<br>(¿Qué se genera?) | -Ruta optimizada para cada camión (lista definitiva)<br>(lista ordenada de contenedores)<br>-Notificaciones a los conductores con la ruta asignada<br>(por app móvil, intranet, o email) |

*2. Diccionario de Datos Conceptual (Ref. 1.2.5)*
*Define las entidades del negocio sin pensar aún en tablas SQL, sino en la "verdad" del negocio.*
*● Entidad: Contenedor.*
*Atributos: ID, Tipo (Vidrio/Orgánico), Capacidad (Litros), Coordenadas (Lat/Long).*
*● Entidad: Ruta de Recogida.*
*Atributos: Fecha, Camión asignado, Lista de paradas, Kilómetros totales.*
*● Instrucción: Debes justificar la Cardinalidad. ¿Un camión puede tener varias rutas? ¿Una ruta puede tener varios camiones?*

A - Principales Entidades

| Entidad | Descripción | Atributos |
|---|---|---|
| Usuario | Es la persona que interactúa con el sistema. Puede ser ciudadano, conductor, administrador. | -ID (identificador personal)<br>-Nombre y apellidos<br>-Correo electrónico<br>-Contraseña (encriptada)<br>-Rol del usuario (ciudadano/conductor/administrador) |
| Ciudadano | Es el usuario que consulta la información y puede reportar incidencias | -ID (identificador personal)<br>-Nombre y apellidos<br>-Correo electrónico<br>-Contraseña (encriptada)<br>-Rol del usuario (ciudadano/conductor/administrador) |
| Conductor | Es el usuario encargado de realizar las rutas de recogida. | -Carnet de conducir (datos del carnet de conducir)<br>-Disponibilidad (turno, días)<br>-Teléfono o Tablet (emergencia, notificaciones) |
| Contenedor | Recipiente cuya función la recogida de residuos de la vía pública | - ID (código único)<br>- Tipo de residuo (vidrio, orgánico, plástico, resto)<br>- Coordenadas (latitud, longitud)<br>- Capacidad (litros, kilos)<br>- Nivel de llenado actual (0%-100%)<br>- Estado operativo (activo,en mantenimiento,averiado) |
| Camión | Vehículo utilizado para la recogida de residuos de contenedores | -Matrícula (identificador único)<br>-Capacidad de carga (kg o litros)<br>-Tipo de actividad (recogida normal, mantenimiento) |
| Ruta | Recorrido predefinido pasando por la lista de contenedores determinados que un conductor debe visitar en ese día. | -ID (código de ruta)<br>-Fecha y hora o turno<br>-Camión asignado<br>-Conductor asignado<br>-Lista ordenada de contenedores (paradas) |
| Incidencia | Reporte de realizado por un usuario (ciudadano o conductor) de un problema en un contenedor | -ID<br>-Fecha y hora<br>-Contenedor afectado<br>-Tipo de incidencia (contenedor lleno, roto, averiado, otro)<br>-Ciudadano que reporta la incidencia<br>-Foto (opcional)<br>-Estado de la incidencia (pendiente, en proceso, resuelta) |

| | | |
|---|---|---|
| Notificación | Mensaje enviado desde el sistema a un usuario (ciudadano o conductor) | -ID<br>-Fecha de envío<br>-Destinatario (usuario)<br>-Tipo de mensaje (app, email)<br>-Contenido (texto) |

B - Relaciones y cardinalidades

| Relación | Cardinalidad | Justificación |
|---|---|---|
| Usuario – Ciudadano | Un usuario - un ciudadano | Cada ciudadano es un tipo de usuario |
| Usuario – Conductor | Un usuario - un conductor | Cada conductor es un tipo de usuario |
| Usuario - Administrador | Un usuario - un administrador | Cada administrador es un tipo de usuario |
| Ruta – Contenedor | Una ruta - muchos contenedores | Dentro de una ruta existen varios contenedores<br>Un contendor puede estar en distintas rutas. |
| Camión – Ruta | Un camión - muchas rutas | Un camión puede realizar varias rutas (días).<br>Cada ruta utiliza un solo camión en un día. |
| Conductor – Ruta | Un conductor - muchas rutas | Un conductor puede realizar varias rutas en diferentes fechas<br>Cada ruta es asignada a un único conductor en una fecha determinada. |
| Ciudadano- Incidencia | Un ciudadano- muchas incidencias | Un ciudadano puede reportar muchas incidencias<br>Cada incidencia pertenece a un único ciudadano. |
| Contenedor – Incidencia | Un contenedor - muchas incidencias | Un contenedor puede tener varias incidencias en diversas fechas<br>Cada incidencia pertenece solo a un contenedor |
| Usuario – Notificación | Un usuario - muchas notificaciones | Un usuario puede recibir varias notificaciones.<br>Cada notificación va dirigida a un único usuario. |

*ENTREGABLE 3: ANÁLISIS ORIENTADO A OBJETOS CON UML*
*(Referencias: 1.2.6 Análisis OO y 1.2.7 Diagramas UML)*
*Aquí es donde demostráis vuestra capacidad de abstracción. Debéis modelar el dominio*
*del problema utilizando el estándar UML.*

*1. Diagrama de Casos de Uso (Ref. 1.2.7)*
*Dibuja las interacciones entre los actores y el sistema.*
*● Actores: Ciudadano, Conductor, Administrador, Sistema de Tráfico externo (API).*
*● Casos de Uso: "Reportar Incidencia", "Optimizar Ruta", "Validar Credenciales".*
*● Detalle: Usa relaciones de <<include>> (para el login obligatorio) y <<extend>> (para casos excepcionales como "Error de GPS").*

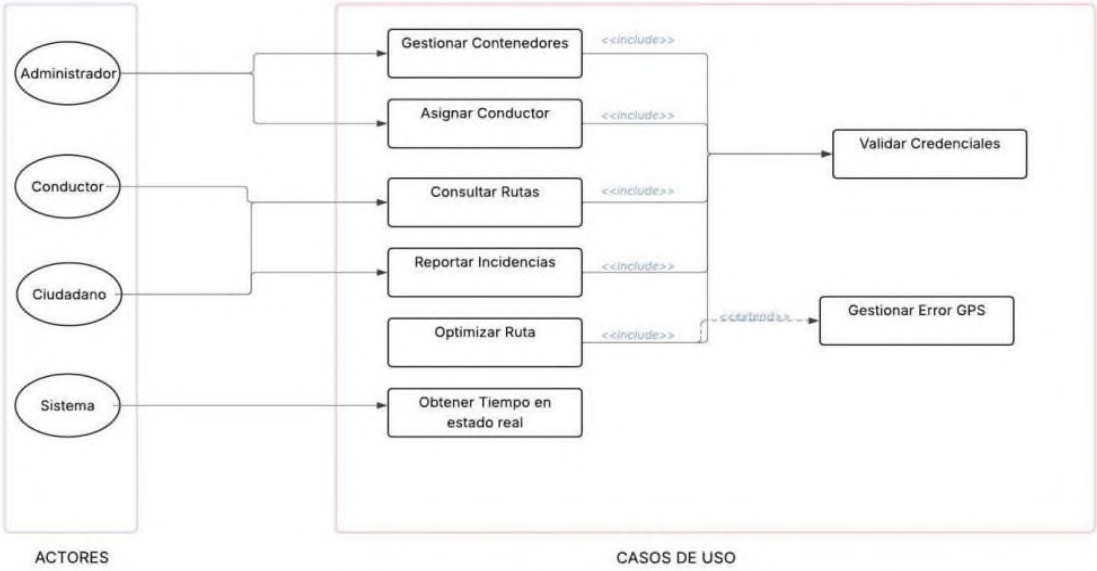
A- Tabla del Diagrama de Casos de Uso

Realizamos el diagrama basado en la siguiente tabla:

| Actor | Descripción |
|---|---|
| Ciudadano | Usuario registrado que puede reportar incidencias y consultar el estado de sus reportes. |
| Conductor | Usuario responsable de realizar las rutas de recogida; recibe rutas optimizadas y actualiza el estado de los contenedores. |
| Administrador | Usuario con privilegios de gestión sobre contenedores, conductores y rutas, así como generación de informes. |

| | | |
|---|---|---|
| Sistema de Tráfico Externo (API) | Servicio externo que proporciona datos de geolocalización, distancias y tiempos reales | |
| Principales Casos de uso | | |
| Actor | Caso de uso | Descripción |
| Sistema | Validar Credenciales | Verifica la identidad del usuario para acceder al sistema. |
| Sistema | Optimizar Ruta | El sistema calcula la ruta optima basándose en el nivel de llenado de los contenedores (>75%) |
| Administrador | Gestionar Contenedores | El administrador da de alta, modifica o elimina contenedores |
| Administrador | Asignar Conductor | El administrador asigna un conductor a una ruta optima |
| Conductor | Consultar Rutas | El conductor visualiza las rutas que tiene asignadas |
| Ciudadano | Reportar Incidencia | El ciudadano reporta la incidencia de un contenedor pudiendo adjuntar foto y comentarios |

B- Diagrama de Casos de Uso




*2. Diagrama de Clases del Dominio (Ref. 1.2.6)*
*Ojo: No es un diagrama de base de datos. Es un diagrama conceptual.*
*● Debes aplicar Herencia: Crea una clase padre Usuario y clases hijas Ciudadano y Empleado.*
*● Debes aplicar Encapsulamiento: Define qué atributos son privados (ej: password) y qué métodos son públicos (ej: getUbicacion()).*
*● Relaciones: Muestra cómo se asocia un Contenedor con un Barrio.*

A- Tabla de Clases según los Atributos

Con el fin de establecer el diagrama de clases del dominio establecemos siguiente tabla de clases según los atributos:

| Clase | Atributos | Métodos públicos |
|---|---|---|
| Usuario | Id<br>nombre<br>email<br>password | login(email, password)<br>getPerfil()<br>getNombre() |
| Ciudadano | direccion<br>telefono | reportarIncidencia(contenedor, tipo, foto) |

| | | |
|---|---|---|
| | | consultarIncidencias() |
| Empleado | numeroEmpleado<br>Rol (conductor/administrador) | accederPanelControl()<br>obtenerRutasAsignadas() |
| Contenedor | Id<br>nivelLlenado (0-100)<br>ubicacion (encoordenadas)<br>barrio | getNivelLlenado()<br>actualizarLlenado(nivel)<br>esPrioritario() |
| Barrio | nombre<br>codigoPostal<br>coordenadas (long;lat) | getNombre()<br>getContenedores() |
| Ruta | id<br>fecha<br>conductor (nomebreempleado)<br>paradas (totaldeparadas) | asignarConductor(conductor)<br>agregarParada(contenedor) |

Atributos privados: en rojo
Atributos protegidos: en naranja
Atributos públicos: en verde

B - Herencia
La herencia nos permite utilizar atributos comunes utilizarlos en otros datos sin tener que repetirlos y evitando duplicidades. Lo mismo ocurre para los métodos.

Tabla de *Herencias* para EcoLogistics Web

| *Clase padre* | *Clase hija* | *Atributos heredados* | *Atributos añadidos* | *Métodos heredados* | *Métodos añadidos* |
|---|---|---|---|---|---|
| *Usuario* | *Ciudadano* | *-Identificador<br>-Nombre<br>-Correo<br>-Contraseña (protegida)* | *-Dirección<br>-Teléfono* | *-Iniciar sesión<br>-Obtener perfil<br>-Obtener nombre* | *-Reportar incidencia<br>-Consultar incidencias* |
| *Usuario* | *Empleado* | *-Identificador<br>-Nombre<br>-Correo<br>-Contraseña (protegida)* | *-Número de empleado* | *-Iniciar sesión<br>-Obtener perfil<br>-Obtener nombre* | |
| *Empleado* | *Conductor* | *-Identificador<br>-Nombre<br>-Correo<br>-Contraseña<br>-Número de empleado* | *-Carné de -conducir<br>-Horario de trabajo* | *-Iniciar sesión<br>-Obtener perfil<br>-Obtener nombre* | *-Consultar rutas asignadas<br>-Actualizar estado de contenedor recogido* |
| *Empleado* | *Administrador* | *-Identificador<br>-Nombre<br>-Correo<br>-Contraseña<br>-Número de empleado* | *-Nivel de permisos* | *-Iniciar sesión<br>-Obtener perfil<br>-Obtener nombre* | *-Gestionar contenedores<br>-Gestionar usuarios<br>-Generar informes<br>-Asignar rutas* |

C - Encapsulamiento

Gracias al encapsulamiento ocultaltamos los atributos de la clase exponiendo tan solo aquello que necesario a través de métodos públicos y protegiendo la integridad de los datos. Del mismo modo facilitamos la interacción y el mantenimiento del sistema.

a) Tabla de Encapsulamiento para EcoLogistics Web

| Clase | Atributos | Métodos públicos |
|---|---|---|
| Usuario | -Id<br>-nombre<br>- email<br>- password | -Iniciar sesión<br>-tipo de perfil(ciudadano, conductor, administrador)<br>-nombre del usuario |
| Administrador | - Número de empleado<br>- Permisos | -Gestionar contenedores(alta, modificar, eliminar)<br>-Gestionar usuarios (ciudadanos y empleados)<br>-Generar informes eficiencia e incidencias<br>-Asignar rutas a los conductores |
| Ciudadano | - direccion<br>- telefono | -Para reportar una incidencia de un contenedor<br>-Consultar el historial de incidencias |
| Conductor | - Número de empleado<br>-Carnet de conducir<br>-Horario/turno | -Para consultar las rutas asignadas<br>-Para marcar un contenedor recogido, o estado |
| Coordenadas | -latitud<br>-longitud | -Obtener la latitud<br>-Obtener la longitud |
| Contenedor | -Id<br>-nivel de llenado<br>(0-100)<br>-ubicacion<br>(en coordenadas)<br>-barrio | -Obtener el nivel de llenado de contenedores |
| Barrio | -nombre<br>-código postal<br>-coordenadas<br>(longuitud;latitud) | -Obtener el nombre del barrio<br>-Obtener la lista de contenedores que pertenecen a ese barrio |
| Ruta | -id<br>-fecha<br>-conductor<br>-paradas | -Asignar un conductor a una ruta<br>-Añadir un contenedor una ruta |
| Incidencia | -id<br>-tipo de incidencia<br>-fecha y hora<br>-foto<br>-estado<br>(pendiente, en proceso, resuelto) | -Obtener todos los detalles de la incidencia (tipo, fecha, foto, estado)<br>-Establecer el estado de la incidencia |

Atributos privados: en rojo
Atributos protegidos: en naranja
Atributos públicos: en verde

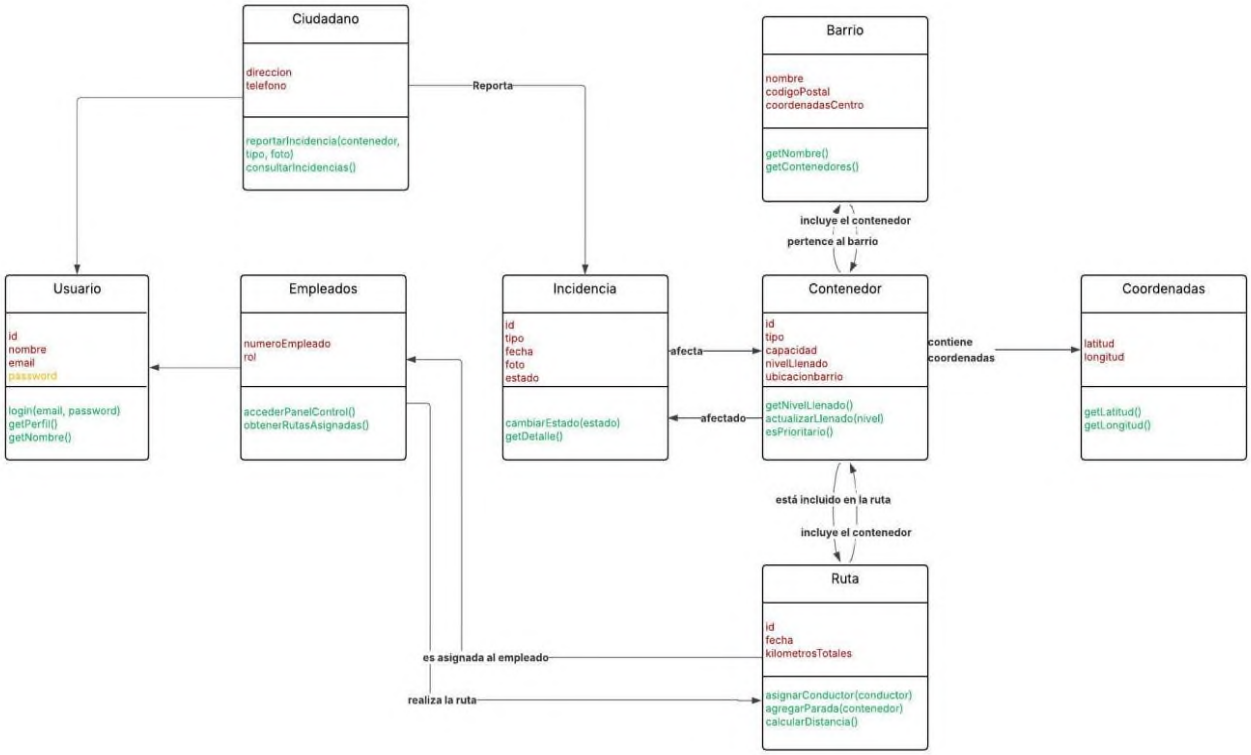
D- Relaciones y Cardinalidad

| ***Relación*** | ***Cardinalidad*** | ***Descripción*** |
|---|---|---|
| ***Contenedor - Coordenadas*** | ***Un contedor- unas cordenadas solo*** | ***Un contenedor tiene unas coordenadas solo. Unas coordenadas especificas corresponden a un contenedor solo.*** |
| ***Contenedor - Barrio*** | ***Muchos contenedores- un barrio*** | ***Un contenedor está en solo barrio. Un barrio tiene muchos contenedores.*** |
| ***Ruta - Contenedor*** | ***Muchas rutas-*** | ***Una ruta incluye muchos contenedores.*** |

| | | |
|---|---|---|
| | ***muchos contenedores*** | ***Un contenedor puede aparecer en varias rutas al mismo tiempo*** |
| ***Contenedor - Incidencia*** | ***Un contenedor-muchas incidencias*** | ***Un contenedor puede tener varias incidencias. Una incidencia puede afectar solo a un contenedor.*** |
| ***Empleado (Conductor) - Ruta*** | ***Un conductor-muchas rutas*** | ***Un conductor puede realizar varias rutas en distintos horarios. Esa ruta es asignada solo a ese conductor.*** |
| ***Ciudadano - Incidencia*** | ***Un ciudadano-varias incidencias*** | ***Un ciudadano puede reportar varias incidencias. Esa incidencia corresponde solo a ese ciudadano.*** |

E - Diagrama de Clases del Dominio

El diagrama de clases del dominio de Ecologistics es la representación gráfica de los principales conceptos del sistema de negocio de la gestión de residuos (ciudadano, contenedor, ruta, etc.), sus correspondientes atributos (ID, nombre, nivel de llenado, etc.), métodos (iniciar sesion, gestionar contenedores, etc.) y las relaciones que existen entre ellos (herencias, encapsulamiento, etc.). En el diagrama de clases podemos ver todas esas relaciones.




***3. Diagrama de Secuencia (Ref. 1.2.7)***
***Elige el caso de uso "Ciudadano reporta contenedor lleno" y dibuja la línea de tiempo.***
- ***Objetos: InterfazCiudadano -> ControladorIncidencias -> EntidadContenedor.***
- ***Muestra los mensajes síncronos y asíncronos.***

***● Objetivo: Demostrar que entendéis que el sistema no es mágico, sino una secuencia de pasos lógicos.***

☑ Mensaje Síncrono
Se dice que un mensaje es síncrono cuando el emisor envía una solicitud y ésta se queda en espera hasta que el receptor procesa la petición y envía la respuesta. Hasta que la respuesta no ha sido recibida por parte del emisor no continúa la ejecución.
Su representación gráfica es una línea continúa y flecha sólida (->); la respuesta es una línea discontinúa (-->).

☑ Mensaje Asíncrono:
Se dice que un mensaje es asíncrono cuando el emisor envía una solicitud y ésta continúa inmediatamente sin esperar respuesta del receptor. No se produce ningún bloqueo en la comunicación. Su representación gráfica es una línea discontinúa y flecha abierta (-); no hay gráfico para la respuesta.

A- Caso del Ciudadano que reporta un contenedor lleno.
Hemos seleccionado el caso de uso "Ciudadano reporta contenedor lleno" con el fin de ilustrar la interacción entre los objetos a lo largo un intervalo de tiempo.

a) Secuencia de tiempos

i. Ciudadano → Interfaz Ciudadano
El ciudadano rellena el formulario y pulsa “Enviar”. (acción del usuario)

ii. InterfazCiudadano → Controlador Incidencias (mensaje síncrono)
La interfaz solicita al controlador de incidencias que cree una incidencia con los datos del formulario que ha recibido y se queda a la espera de la respuesta.

iii. Controlador Incidencias → Entidad Contenedor (mensaje síncrono)
El controlador pregunta a la entidad Contenedor si el contenedor indicado existe y si está operativo. Hasta obtener la respuesta el controlador se bloquea.

iv. Entidad Contenedor → Controlador de Incidencias (respuesta de la entidad contenedor)
La entidad Contenedor proporciona la información del contenedor (si existe) o un valor nulo (si no existe).

v. Controlador Incidencias (decisión)
Evalúa si el contenedor es válido (si el contendor existe y corresponde con el indicado).

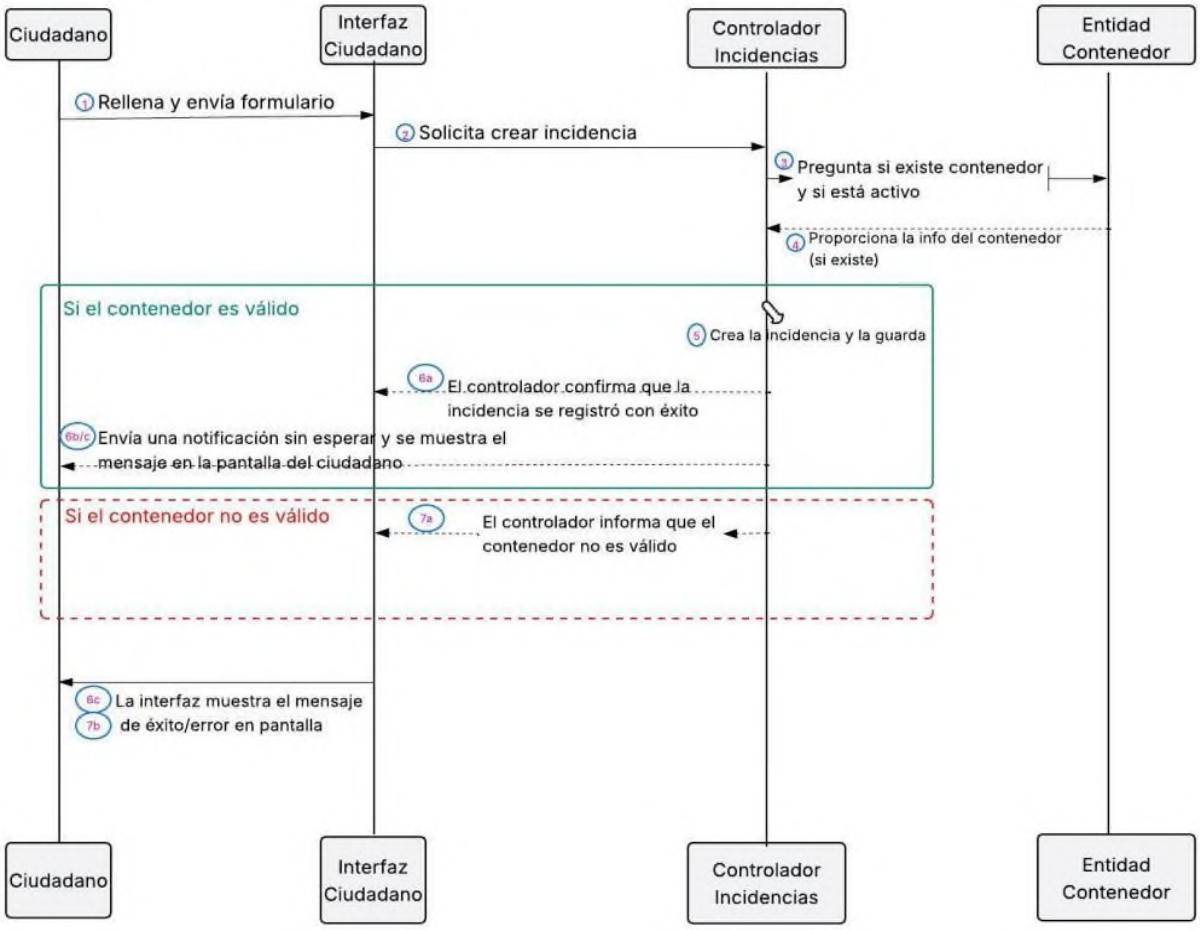
vi. Si es válido:
a. Controlador Incidencias → Interfaz Ciudadano (mensaje síncrono)
El controlador Incidencias confirma a la interfaz que la incidencia se ha registrado con éxito.
b. Controlador Incidencias → Ciudadano (mensaje asíncrono)
El controlador envía una notificación (correo) sin esperar a que se complete.
c. Interfaz Ciudadano → Ciudadano
La interfaz muestra el mensaje de éxito en pantalla.

vii. Si no es válido:
a. Controlador Incidencias → Interfaz Ciudadano (mensaje síncrono)
El controlador informa a la interfaz de que el contenedor no es válido.

b. Interfaz Ciudadano → Ciudadano
La interfaz muestra el mensaje de error en pantalla.

b) Diagrama de Secuencia

El diagrama muestra la secuencia de mensajes (síncronos y asíncronos) desde el reporte de la incidencia por el ciudadano hasta que el sistema confirma la recepción de la misma por un mensaje de éxito o error en la pantalla del ciudadano.




*INSTRUCCIONES PARA EL JEFE DE PROYECTO (ALUMNO)*
*Para realizar esta entrega, debes gestionar a tus 21 recursos simulados de la siguiente manera:*
*1. Asignación de Tareas:*
*Asigna a los Perfiles Analíticos (Seniors) la creación del Diagrama de Clases y Secuencia, ya que requieren mayor abstracción (1.2.6).*
*Asigna a los Perfiles Medios la redacción detallada de los Requisitos (ERS) y la Matriz de Trazabilidad (1.2.2 y 1.2.3).*
*Asigna a los Perfiles Junior la elaboración del Diccionario de Datos y el listado de Casos de Uso simples, bajo supervisión.*

1. Organización del Equipo por Perfiles
Para abordar las tareas de diseño de interfaz, lógica detallada y arquitectura, he distribuido el trabajo según las capacidades de cada perfil, garantizando que quienes

están empezando aprendan desde la perspectiva del usuario y que los perfiles más avanzados aporten visión técnica.

| Perfil | Tarea asignada | Justificación |
|---|---|---|
| Trainees | Casos de Uso simples y Diccionario de Datos Conceptual con acompañamiento (ref. 1.2.5 y 1.2.7) | Pese a tener poca experiencia técnica pueden identificar bien las entidades del dominio del diagrama de casos y describir sus atributos básicos intuitivamente con acompañamiento. Este tipo de tareas facilita el análisis de un nivel alto de abstracción. |
| Juniors | Redacción detallada de la Especificación de Requisitos Software (ERS), incluyendo el Catálogo de Requisitos y la Matriz de Trazabilidad ( ref. 1.2.2 y 1.2.3) | Ya familiarizados con los requisitos funcionales y no funcionales poseen la capacidad de redactar documentación formal. La matriz de trazabilidad facilita su análisis y la compresión de los requisitos. |
| Seniors | Diagrama de Clases UML y Diagrama de Secuencia (ref. 1.2.6 y 1.2.7) | Los diagramas de Clases y de Secuencia requieren de una comprensión profunda de la estructura estática del sistema y de la dinámica temporal entre objetos. Los perfiles Senior garantizan coherencia y solidez. Además actúan como mentores para los perfiles junior y tranees, guiandolos en la elaboración del diccionario de datos y los casos de uso, y asegurandose que la documentación sea correcta y compatible dentro del proyecto. |
| Proyect Manager | Proyecto | Debe revisar la coherencia del proyecto hasta el momento antes de entregarlo al equipo de desarrollo. |

ANEXO - INFORME DE VALIDACIÓN DE COHERENCIA

Proyecto: EcoLogistics Web – Fase 2.0
Referencia: 1.2.7 (Diagramas UML)
Fecha: 12/01/2026

| ID | Verificación | Resultado |
|---|---|---|
| V-01 | RF-005 (Gestión de contenedores) tiene correspondencia en el diagrama de clases: existen las clases Contenedor, Barrio y Administrador con los atributos y métodos necesarios para el alta, modificación y eliminación. | Conforme |
| V-02 | El diagrama de secuencia (caso "Ciudadano reporta contenedor lleno") refleja los pasos descritos en los requisitos funcionales RF-001 (geolocalización), RF-002 (reporte) y RF-007 (historial). La interacción entre los objetos sigue la lógica de verificación del contenedor antes de crear la incidencia. | Conforme |
| V-03 | No existen contradicciones entre los requisitos no funcionales (rendimiento, seguridad, escalabilidad) y las decisiones de modelado reflejadas en los diagramas UML. | Conforme |
| | | |

Declaración de conformidad:

Por la presente, yo, Angelina, en calidad de Project Manager, declaro que he revisado personalmente los elementos anteriores y confirmo que existe coherencia entre la Especificación de Requisitos Software (ERS) y los diagramas UML (clases y secuencia) desarrollados en el análisis orientado a objetos.

Firma: __Angelina Caballero_____

Fecha: __12/01/2026_____

Curso:  Metodología y gestión de proyectos de Ingeniería Software (IS)

Proyecto: EcoLogistics Web (Sistema Inteligente de Gestión de Residuos)

Responsable Académico: José Luis Martínez Vila

Responsable de Ejecución: Angelina Caballero (Rol:Estudiante/Project Manager)

Semana:  3 Año: 2026

# TABLA DE CONTENIDOS

*FASE 3: DISEÑO DEL SISTEMA ECOLOGISTICS WEB* ..... 1
*Tarea 1: La Arquitectura General (Nivel Alto)* ..... 1
1. Analogía del Restaurante ..... 1
A - Frontend (La vista) -> El móvil del ciudadano ..... 2
B- Backend (Controlador) -> Servidor del Ayutamiento ..... 2
2. Representación esquématica del modelo MVC para Ecologistics Web ..... 2
A- Secuencia del flujo de mensajes: ..... 2
B- Comunicaciones ..... 2
*Tarea 2: Diseño de la Interfaz (UI/UX)* ..... 3
1. Objetivo del diseño de la Interfaz (UI/UX) ..... 3
2. Pantalla de Inicio (Ciudadano) ..... 3
3. Formulario de Incidencia (Ciudadano) ..... 4
4. Principios de Diseño y Criterios de Aceptación ..... 5
*Tarea 3: Diseño Detallado y Principios (El "Paso a Paso")* ..... 5
1. Aplicación del principio de modularidad ..... 6
2. Diagrama visual del flujo ..... 6
3. Procesar el reporte de una incidencia y almacenarlo en el sistema ..... 6
4. Mensajes de Error ..... 7
INSTRUCCIONES PARA LA GESTIÓN DEL EQUIPO ..... 7
1. Organización del Equipo por Perfiles ..... 7

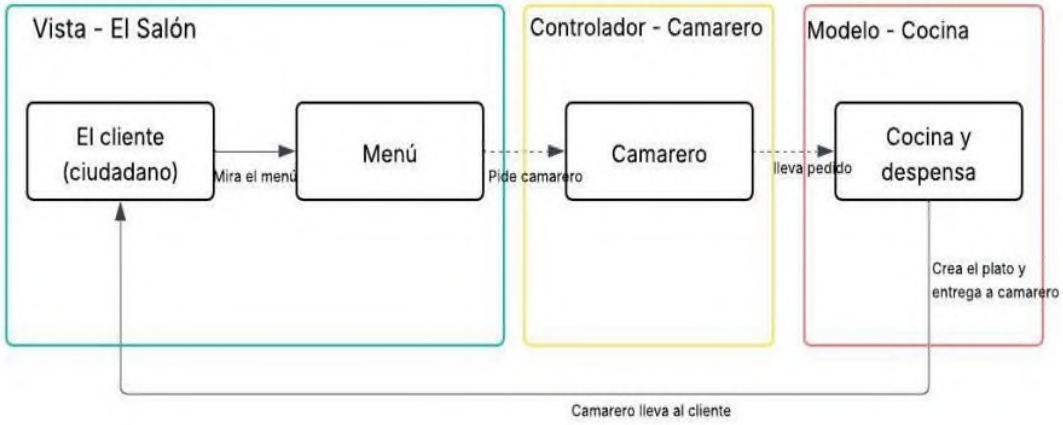
**Semana 3 – DISEÑO DEL SISTEMA ECOLOGISTICS WEB**

*FASE 3: DISEÑO DEL SISTEMA ECOLOGISTICS WEB*
*Objetivo: Crear los "planos de la casa" antes de poner los ladrillos (el código).*
*Documentación base: Bloque 1.3 al 1.3.6 del temario ADGG086PO.*
*Para superar esta fase, cada equipo deberá entregar un Dossier de Diseño muy visual, compuesto por las siguientes tres tareas:*

*Tarea 1: La Arquitectura General (Nivel Alto)*
*(Ref: 1.3.1 Enfoques, 1.3.2 Niveles y 1.3.5 Diseño Arquitectónico)*
*Olvidémonos de palabras complejas. La arquitectura es simplemente decidir cómo se reparten las tareas en nuestro sistema. Para EcoLogistics, usaremos el enfoque más clásico y efectivo para la web: la separación en capas o Modelo-Vista-Controlador (MVC).*
*Pensad en un restaurante:*
*● La Vista (Frontend): Es el menú y el salón. Lo que el ciudadano ve y toca en su móvil.*
*● El Controlador (Backend): Es el camarero. Toma la orden del ciudadano y la lleva a la cocina.*
*● El Modelo (Base de Datos): Es la cocina y la despensa. Donde se guardan los datos reales de los contenedores.*
*● Vuestra Misión:*
*Dibujad un esquema muy sencillo (podéis usar cajitas y flechas en herramientas gratuitas como Draw.io, Canva o incluso en papel y sacarle una foto) que muestre estas tres partes conectadas.*
*● Caja 1: Móvil del ciudadano (Frontend).*
*● Caja 2: Servidor del Ayuntamiento (Backend).*
*● Caja 3: Base de datos de contenedores.*

1. Analogía del Restaurante
Para el sistema EcoLogistics seguimos el modelo de arquitectura Modelo-Vista-Controlador (MVC), basado en la separación de tres capas del exterior hacia el interior: Frontend; Backend; Base de Datos.
Utilizamos como ejemplo de similitud un restaurante para mostrar las tres capas o partes conectadas siendo:
- Front-end: El móvil del ciudadano
- Backend: El sevidor del Ayutamiento
- La Base de Datos de contenedores : Caja 3

Representación esquématica del modelo MVC - ejemplo Restaurante



A - Frontend (La vista) -> El móvil del ciudadano

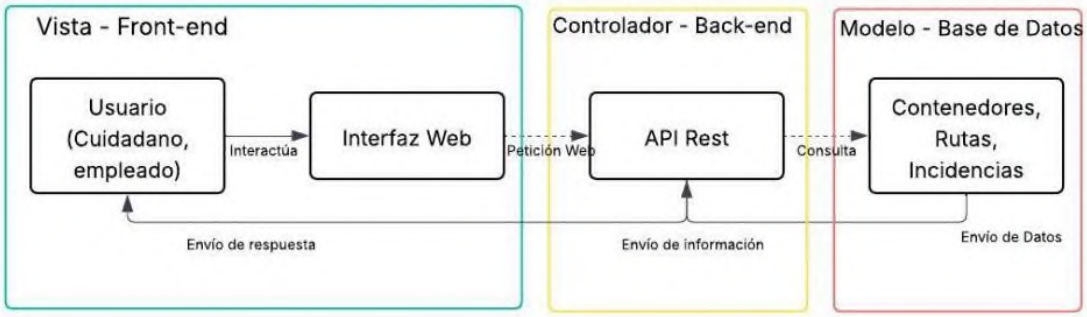
- El móvil del ciudadano es como el menú y el salón del restaurante.
- Es lo que el usuario vey con lo que interactúa: el mapa, los botones, los formularios, información de los contenedores.
- Cuando el usuario (ciudadano o conductor) toca la pantalla para reportar una incidencia está haciendo lo mismo que un cliente en el restaurante al levantar la mano para pedir al camarero su consumición.

B- Backend (Controlador) -> Servidor del Ayutamiento

- Aquí el backend actúa como lo haría el camarero al recibir la solicitud del usuario (ciudadano o conductor). Interpreta la petición y decide que va a hacer con ella, luega la lleva a la cocina. La cocina corresponde a la base de datos o servidor del ayuntamiento.
- Cuando la cocina tiene el plato preparado lo envía de nuevo a ciudadano a través del camarero. En este caso la base de datos envía la respuesta, y es el backend quien proporciona la respuesta al ciudadano.

Cada parte o cada caja tiene su rol y se comunican entre ella de manera ordenada. De la misma manera que un cliente nunca entra en la cocina del restaurante, el ciudadano nunca habla directamente con la base de datos.

2. Representación esquématica del modelo MVC para Ecologistics Web




A- Secuencia del flujo de mensajes:

1) El usuario interactúa con la interfaz web (Front-end)
2) La Interfaz situada en el Front-end envía una petición al Controlador situado en el Back-end
3) El Controlador procesa la petición y aplica reglas de negocio.
4) Envía una consulta o modificación a la base de datos o el Modelo
5) El devuelve los datos al Controlador.
6) El Controlador envía la respuesta al Front-end o Vista con los datos.
7) La Vista actualiza la pantalla y muestra la respuesta definitiva al usuario.

B- Comunicaciones

a) Usuario → Servidor

Lenguajes/formato: x
Protocolo: HTTPS

b) Frontend → Backend

Lenguajes/formato: JSON & JavaScript
Protocolo: HTTPS

c) Backend → Frontend

Lenguajes/formato: JSON

Protocolo: HTTPS

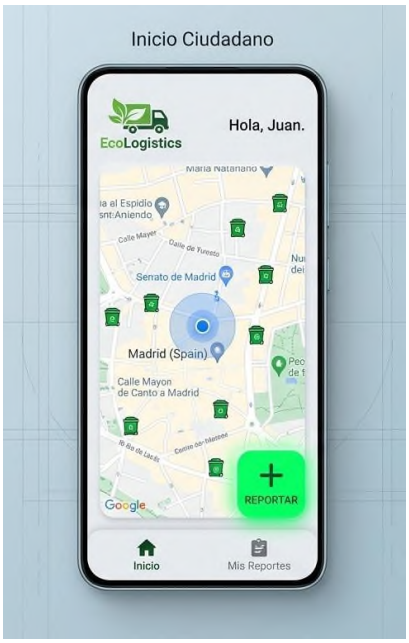
d) Backend → Base de Datos
Lenguajes/formato: SQL
Protocolo:x

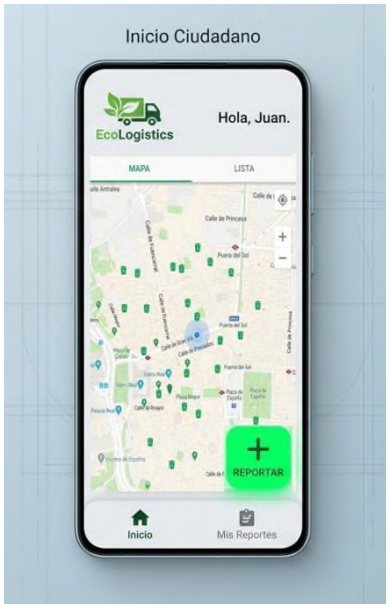
*Tarea 2: Diseño de la Interfaz (UI/UX)*
*(Ref: 1.3.4 Diseño de la Interfaz de Usuario)*
*Esta suele ser la parte favorita de los equipos. El diseño de la Interfaz de Usuario (UI) es cómo se ve la pantalla, y la Experiencia de Usuario (UX) es lo fácil que es usarla. Si un abuelo no puede usar nuestra app para avisar de que un contenedor está lleno, hemos fracasado como ingenieros, por mucho código que haya detrás.*
*Vuestra Misión:*
*Vais a crear Mockups (bocetos visuales) de las dos pantallas principales de la aplicación. Podéis usar lápiz y papel o herramientas intuitivas como Balsamiq, Figma (nivel básico) o PowerPoint.*
*1. Pantalla de Inicio (Ciudadano): ¿Dónde va el botón para reportar una incidencia? ¿Hay un mapa?*
*2. Formulario de Incidencia: ¿Qué datos pedimos? (Foto, calle, tipo de basura).*
*Requisito de calidad: Aplicad el principio de simplicidad. Los botones deben ser grandes (pensando en móviles) y los textos muy claros*

1. Objetivo del diseño de la Interfaz (UI/UX)

- El diseño de la interfaz (UI/UX) busca ofrecer una experiencia de usuario basada en la accesibilidad y simplicidad adaptada para dispositivos móviles.
- Se ha priorizado la eficacia (solo los elementos esenciales), botones grandes (fáciles de pulsar con el pulgar) y textos claros (sin tecnicismos).
- A continuación se presentan los mockups de las dos pantallas principales del ciudadano, desarrolladas siguiendo los principios de UI/UX.

2. Pantalla de Inicio (Ciudadano)




Figuras 1 y 2 – Pantalla de inicio del ciudadano

Tabla del contenido de la pantalla de Incio del Ciudadano

| | | |
|---|---|---|
| Encabezado y saludo | “EcoLogistics” y “Hola, Juan”. | Personaliza la experiencia y orienta al usuario. |
| Mapa interactivo | Vista del mapa con contenedores cercanos, calles (Calle Mayor, Calle de Fuencarral, etc.) y ubicación actual. | El mapa ocupa la mayor parte de la pantalla, permitiendo al ciudadano visualizar rápidamente los contenedores y su estado.<br>Es la herramienta principal para decidir dónde reportar. |
| Botón “REPORTAR” | Botón grande, verde, centrado en la parte inferior, con texto en mayúsculas | Cumple el principio de simplicidad: es la acción principal y está diseñado para ser pulsado con el pulgar (zona de confort táctil). Su color contrasta con el fondo, facilitando su localización. |
| Menú inferior | Iconos “Inicio” (activo) y “Mis Reportes”. | Permite navegar al historial de incidencias sin saturar la pantalla. El icono de inicio indica la pantalla |

La pantalla se ha diseñado para que el usuario, en menos de un segundo, identifique dónde está, dónde están los contenedores y cómo reportar un problema.

El botón de reporte es el elemento más destacado, reforzando la acción principal del sistema.

3. Formulario de Incidencia (Ciudadano)

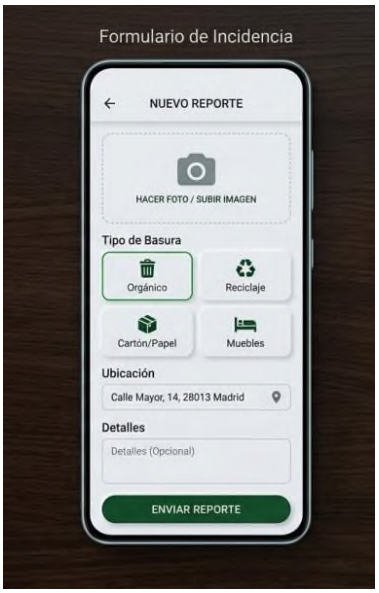



Figura 3 – Formulario de incidencia

| Campo / Elemento | Descripción | Cumplimiento de requisitos |
|---|---|---|
| Título“NUEVO REPORTE”<br><br>Botón “HACER FOTO/SUBIR IMAGEN”<br><br>Tipo de Basura | Encabezado claro que indica la acción en curso.<br><br>Botón grande, con icono de cámara y texto descriptivo.<br><br>Botones grandes, iconos y texto: Orgánico, Reciclaje, Cartón/Papel, Muebles. | Ayuda al usuario a situarse dentro del flujo.<br>Permite adjuntar evidencia visual de forma sencilla.<br>Es opcional, pero muy útil para el equipo d mantenimiento<br>Opciones táctiles que reducen la carga cognitiva. Los iconos ayudan a identificar rápidamente la categoría sin |

| | | |
|---|---|---|
| Ubicación | Campo pre-rellenado: “Calle Mayor, 14, 28013 Madrid”. | necesidad de leer texto largo. Se obtiene automáticamente del contenedor seleccionado en el mapa, evitando que el usuario tenga que escribir la dirección. |
| Detalles (Opcional) | Campo de texto grande. | Permite añadir información adicional sin obligar al usuario. |
| Botón “ENVIAR REPORTE” | Botón grande, verde, al final del formulario. | Acción final clara y destacada. Una vez pulsado, el sistema muestra una confirmación y vuelve al mapa. |

- ☑ El formulario recoge solo la información indispensable para gestionar la incidencia.
- ☑ Los botones de tipo de basura son grandes y están etiquetados con palabras comunes, lo que facilita la selección incluso para personas con poca experiencia digital.
- ☑ La ubicación se autocompleta, reduciendo la entrada manual y los posibles errores.

4. Principios de Diseño y Criterios de Aceptación

| Principio/ Criterio | Descripción |
|---|---|
| Simplicidad | Ambas pantallas contienen solo los elementos necesarios.<br>No hay menús desplegables complejos ni campos redundantes. |
| Botones grandes y accesibles | Todos los botones principales (REPORTAR, HACER FOTO, tipos de basura, ENVIAR REPORTE) tienen un tamaño mínimo de 44×44 píxeles, apto para pulsación con dedo. |
| Textos claros | Se utilizan palabras directas: “REPORTAR”, “NUEVO REPORTE”, “HACER FOTO / SUBIR IMAGEN”, “ENVIAR REPORTE”.<br>Los textos de ayuda son cortos y evitan jerga técnica. |
| Jerarquía visual | El mapa y el botón de reporte dominan la pantalla de inicio.<br>En el formulario, los botones se presentan en cuadrícula para facilitar la elección. |
| WCAG 2.1 AA | Contraste de color suficiente: El texto oscuro sobre fondo claro o botón verde sobre fondo gris cumple con las relaciones de contraste mínimas. |
| Flujo reporte ≤3 Interacciones | 1. Pulsar “REPORTAR”<br>2. SELECCIONAR Tipo de basura<br>3. Pulsar “ENVIAR REPORTE” |

***Tarea 3: Diseño Detallado y Principios (El "Paso a Paso")***
***(Ref: 1.3.3 Principios y 1.3.6 Diseño Detallado)***
***Uno de los principios clave del diseño (1.3.3) es la Modularidad: dividir un problema grande en problemas pequeños. No vamos a diseñar toda la lógica de golpe. Vamos a hacer el diseño detallado (1.3.6) de una sola acción pequeña.***
***Vuestra Misión:***
***Elegid el botón "Enviar Incidencia" de vuestro boceto visual. Escribid, como si fuera una receta de cocina en lenguaje normal (sin código complejo), qué pasos lógicos debe dar el sistema cuando el usuario pulsa ese botón.***
***Ejemplo de la "receta":***

*1. El sistema comprueba que el usuario ha subido una foto y escrito una dirección. (Si falta algo, muestra un error en rojo).*
*2. El sistema (Camarero/Controlador) coge esos datos y los envía a la Base de Datos (Cocina).*
*3. La Base de Datos guarda la incidencia con el estado "Pendiente".*
*4. La pantalla muestra un mensaje verde: "¡Gracias por avisar!".*

1. Aplicación del principio de modularidad

- En lugar de construir todo el sistema de una sola vez, aplicamos el principio de modularidad (ref. 1.3.3) descomponemos el problema general en elementos más manejables.
- Para el botón "Enviar Incidencia" del formulario de reporte de incidencias, preparamos una guía con los pasos que debe seguir el sistema cuando el usuario lo pulsa. Este diseño detallado (ref. 1.3.6) servirá para que los desarrolladores puedan implementarlo sin margen de interpretación.

2. Diagrama visual del flujo

La siguiente figura muestra de forma esquemática la interacción entre el ciudadano, el servidor del ayuntamiento y la base de datos al pulsar el botón "Enviar Incidencia".

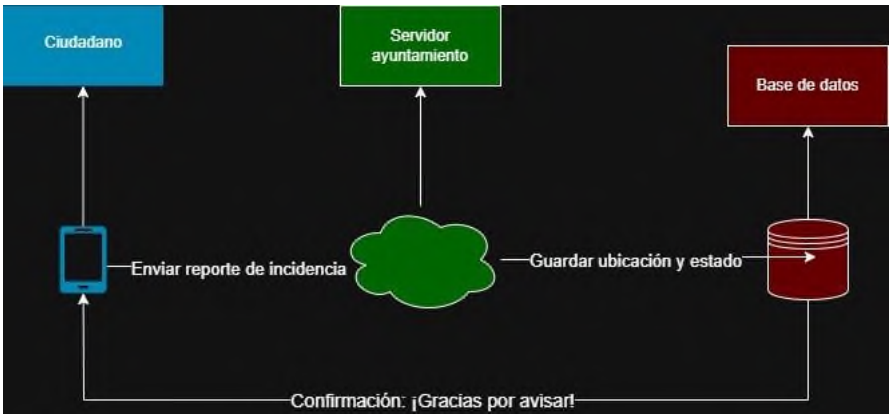



Figura 1 – Flujo de procesamiento del botón "Enviar Incidencia"

Pasos del Flujo de procesamiento del botón "Enviar Incidencia" - Figura 1

| | |
|---|---|
| Ciudadano | El ciudadano pulsa el botón en la pantalla móvil |
| Servidor ayuntamiento | El backend (API REST) recibe y procesa la petición de "enviar reporte de incidencia" |
| Enviar reporte de incidencia | Es la acción que inicia el flujo de procesamiento. |
| Guardar ubicación y estado | Se guardan los datos en la base de datos. |
| Base de datos | Se almacena la incidencia. El estado de la incidencia pasa a ser "pendiente". |
| Confirmación: ¡Gracias por avisar! | Se envía un mensaje confirmación a la interfaz del ciudadano. El mensaje recibido por el ciudadano es de éxito por el procesamiento de la incidencia. |

3. Procesar el reporte de una incidencia y almacenarlo en el sistema

Paso a Paso "estilo receta"

1) El sistema recibe la señal de que el usuario ha pulsado "Enviar Incidencia"
2) El sistema comprueba que los datos obligatorios están presentes.

3) Si todo está correcto, el sistema (el "camarero" – Controlador) toma esos datos y los envía a la base de datos (la "cocina").
4) La base de datos guarda la incidencia y confirma que se ha almacenado correctamente. La base de datos guarda la incidencia y confirma que se ha almacenado correctamente.
5) El sistema muestra un mensaje de éxito al ciudadano.
6) El sistema limpia el formulario y devuelve al ciudadano a la pantalla de inicio (mapa), para que pueda realizar otra acción si lo desea.

4. Mensajes de Error

☑ "Error de conexión. Comprueba internet"
Este mensaje aparece cuando existe un problema de red o de servidor. Sucede cuando el sistema no puede conectar con el back-end

☑ " Error del Sistema"
Este mensaje corresponde a un error interno del servidor. Sucede cuando la base de datos falla y el sistema registra internamente el fallo para que el equipo técnico pueda revisarlo.

☑ "Por favor, selecciona el tipo de basura"
Este mensaje corresponde a una falta de datos obligatorios del formulario. Sucede cuando faltan datos y el sistema solicita claramente los datos que necesita.

## INSTRUCCIONES PARA LA GESTIÓN DEL EQUIPO

**Como Jefes de Proyecto, organizad a vuestros "empleados" así:**
**Poned a los perfiles con nivel bajo a dibujar los bocetos de las pantallas(Tarea 2) poniéndose en la piel del usuario.**
**● Poned a los perfiles medios a redactar la lógica paso a paso (Tarea 3).**
**● Poned a los perfiles avanzados a hacer el diagrama de arquitectura y supervisar que todo tenga sentido (Tarea 1).**

1. Organización del Equipo por Perfiles
Para abordar las tareas de diseño de interfaz, lógica detallada y arquitectura, he distribuido el trabajo según las capacidades de cada perfil, garantizando que quienes están empezando aprendan desde la perspectiva del usuario y que los perfiles más avanzados aporten visión técnica.

| Perfil | Tarea asignada | Justificación |
|---|---|---|
| Trainees | Tarea 2: Bocetos de pantallas (UI/UX) – Elaboración de wireframes para la aplicación móvil del ciudadano y del conductor | Su menor exposición a patrones técnicos les permite pensar como usuarios reales. Ponerse en lugar del ciudadano o del conductor favorece la creaciñon de interfaces claras y accesibles. También es una actividad visual y creativa que no exige grandes conocimientos de programanción. |
| Juniors | Tarea 3: Lógica paso a paso (diseño detallado) – Describir la secuencia interna del sistema al enviar una incidencia o realizar el login del conductor. | Los miembros juniors manejan conceptos básicos de lógica, pero todavía necesitn reforzar su capacidad de estructurar procesos antes de escribir código. Redactar el comportamiento del sistema paso a paso les prepara para implementar estructuras condicionales y bucles. |
| Seniors | Tarea 1: Diagrama de arquitectura general (MVC) – Representar las capas del sistema (Vista, | El diagrama MVC requiere una visión completa del sistema y un nivel alto de abstracción técnica. Los seniors deben definir la distribución entre el |

| Perfil | Tarea asignada | Justificación |
|---|---|---|
| | Controlador, Modelo) y asegurar la coherencia global. | frontend, el backend y la base de datos asi como verficar que los bocetos y la lógica se ajusten correctamente a la arquitectura.<br>Además cada senior debe actuar como mentor para trainees y juniors con el fin de fomentar el aprendizaje guiado y la calidad del proyecto. |
| Proyect Manager | Proyecto<br>Diagrama arquitectónico, Wiframe y lógica detallada | Debe revisar la coherencia del proyecto hasta el momento antes de entregarlo al equipo de desarrollo. |

Curso:  Metodología y gestión de proyectos de Ingeniería Software (IS)

Proyecto: EcoLogistics Web (Sistema Inteligente de Gestión de Residuos)

Responsable Académico: José Luis Martínez Vila

Responsable de Ejecución: Angelina Caballero (Rol:Estudiante/Project Manager)

Semana:  4 Año: 2026

# TABLA DE CONTENIDOS

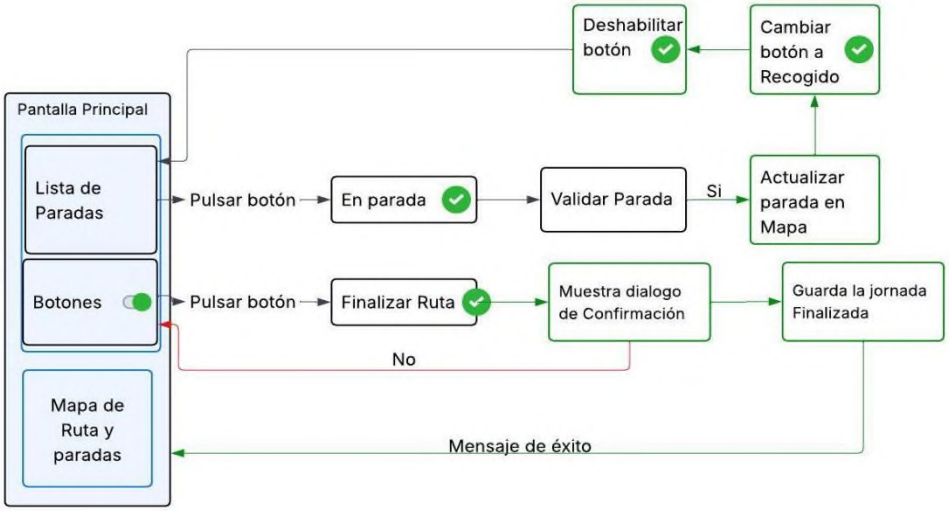
*FASE 3 (Parte II): DISEÑO VISUAL, ESTRUCTURA Y VALIDACIÓN* ..... 1
*Tarea 1: El Diseño de la Interfaz (UI/UX) - Ref. 1.3.4* ..... 1
1. Diseño del Wireframe del conductor ..... 1
*Tarea 2: El Diseño Arquitectónico - Ref. 1.3.5* ..... 2
1. ADR — Elección del Tipo de Base de Datos para EcoLogistics ..... 2
1. Contexto ..... 2
2. Decisión ..... 2
3. Alternativas Consideradas ..... 2
4. Justificación ..... 3
5. Consecuencias ..... 3
6. Estado ..... 3
*Tarea 3: El Diseño Detallado - Ref. 1.3.6* ..... 3
1. Tabla del Proceso de autenticación del conductor ..... 3
2. Tabla de Notas de Seguridad ..... 4
3. Diagrama de Flujo del Proceso de autenticación del conductor ..... 5
*Tarea 4: Técnicas de Diseño y Validación - Ref. 1.3.7* ..... 6
1. Revisión por pares (Peer Review) – Autocrítica del wireframe del conductor ..... 6
a) Mapa demasiado pequeño para la interacción táctil. ..... 6
b) No hay modo sin conexión (offline) ..... 6
c) Contraste insuficiente para conducción diurna ..... 6
*INSTRUCCIONES DE ORGANIZACIÓN PARA TU EQUIPO* ..... 7
1. Organización del trabajo por perfiles ..... 7

**Semana 4 – DISEÑO VISUAL, ESTRUCTURA Y VALIDACIÓN DEL SISTEMA ECOLOGISTICS WEB**

*FASE 3 (Parte II): DISEÑO VISUAL, ESTRUCTURA Y VALIDACIÓN*
*Proyecto: EcoLogistics Web*
*Documentación base: Puntos 1.3.4 a 1.3.7 del temario ADGG086PO.*
*Vuestro Dossier de Diseño debe completarse con las siguientes cuatro tareas prácticas.*
*Como siempre, os lo explico con ejemplos del día a día.*

*Tarea 1: El Diseño de la Interfaz (UI/UX) - Ref. 1.3.4*
*(El escaparate de la tienda)*
*Si la aplicación es difícil de usar, nadie reportará contenedores llenos. El diseño centrado en el usuario (UX) significa ponernos en los zapatos del ciudadano o del conductor del camión.*
*Vuestra Misión:*
*Cread un Boceto Visual (Wireframe) de la pantalla que usará el conductor del camión de basura. No hace falta que tenga colores perfectos, usad formas simples (cuadrados para botones, líneas para textos).*
*● Requisito de Accesibilidad (WCAG): Explicad por escrito por qué habéis puesto los botones grandes (ej."Para que el conductor pueda pulsarlos fácilmente mientras lleva guantes de trabajo").*

1. Diseño del Wireframe del conductor




## Notas de Accesibilidad:

- ✓ Los botones ✓ y FINALIZAR RUTA tienen un tamaño mínimo de 44×44 píxeles (diseñados para pulsación con guantes). Botones grandes: Tanto los ✓ como "FINALIZAR RUTA" superan los 44×44 px, cumpliendo la recomendación WCAG 2.1 para objetivos táctiles.

- ✓ Contraste: El mapa utiliza colores de alto contraste (rojo sobre fondo claro) y los botones tienen bordes definidos para ser visibles incluso con luz solar directa. El contraste de colores en el mapa y la lista se ha verificado para ser visible bajo luz solar directa.

- ✓ Los textos de las calles y porcentajes tienen al menos 16 píxeles para facilitar la lectura desde la cabina del camión. Texto legible: Los nombres de las calles y los

porcentajes de llenado están en tamaño mínimo 16 px, permitiendo su lectura desde la distancia típica de la cabina del camión

- ✓ Feedback claro: Cada acción del conductor produce una respuesta visual inmediata (cambio de color en el mapa, deshabilitación del botón), reduciendo la carga cognitiva.

*Tarea 2: El Diseño Arquitectónico - Ref. 1.3.5*
*(Los cimientos del edificio)*
*La arquitectura del software es decidir qué piezas grandes forman nuestro sistema y cómo se hablan entre ellas. No os compliquéis con diagramas técnicos complejos. Vamos a usar un formato muy sencillo llamado ADR (Registro de Decisión Arquitectónica). Es simplemente dejar por escrito por qué tomamos una decisión.*
*Vuestra Misión:*
*Redactad un pequeño documento de decisión (ADR) para la base de datos de EcoLogistics. Seguid esta plantilla exacta:*
*● Título: Elección del tipo de Base de Datos.*
*● Contexto: Tenemos que guardar datos de ciudadanos, incidencias y rutas de camiones que están muy relacionados entre sí.*
*● Decisión: (Elegid una: Base de datos Relacional con tablas clásicas o NoSQL como MongoDB).*
*● Razón: (Explicadlo de forma sencilla, ej: "Elegimos tablas clásicas porque las incidencias siempre estarán unidas a un usuario y a un contenedor concreto")*
*Analogía del Restaurante*

1. ADR — Elección del Tipo de Base de Datos para EcoLogistics

Título: Elección de Base de Datos Relacional para el Sistema de Gestión de Residuos EcoLogistics (Ciudadanos, Incidencias, Rutas y Contenedores)

1. Contexto
Ecologistics Web constituye una solución integral para la gestión de residuos urbanos, que se centraliza en una plataforma de tres ámbitos claros: la comunicación de incidencias por parte de los ciudadanos, la optimización de las rutas de recogida para los conductores y la administración de contenedores y operaciones logísticas por parte del personal gestor.

La información crítica controlada por el sistema es:
☑ Ciudadanos (datos personales, reportes de incidencias)
☑ Contenedores (ubicación, tipo, nivel de llenado, estado operativo)
☑ Incidencias (reportes de contenedores llenos, rotos, etc.)
☑ Rutas logísticas (asignación de conductores, orden de paradas, tiempos)
☑ Conductores y vehículos (disponibilidad, turnos, capacidad)

2. Decisión
Se selecciona una base de datos relacional (RDBMS) como tecnología principal para la persistencia de la información para la plataforma EcoLogistics Web.

3. Alternativas Consideradas
☑ Base de datos NoSQL (MongoDB)
Ofrece flexibilidad en el esquema y escalabilidad horizontal sencilla, pero no proporciona integridad referencial nativa ni un modelo óptimo para consultas compljas.
☑ Base de datos relacional (PostgreSQL, MySQL, SQL Server)
Proporciona relaciones estructuradas, trasacciones ACIS, integridad referencial y un lenguaje de consultas robusto (SQL), aducuado para operaciones críticas y auditoría.

4. Justificación
La elección del modelo relacional se fundamenta en los siguientes motivos:
- ☑ Las entidades del dominio presentan relaciones fuertes y bien definidas ( un ciudadano reporta incidencias, un camión recoge múltiples contenedores, cada incidencia pertenece a un contenedor concreto).
- ☑ Las operaciones clave del negocio requieren consitencia estricta: creación de incidencias, actualización del nivel de llenado, asignación de rutas.
- ☑ El sistema debe ejecutar consultqas complejas y frecuentes, incluyendo joins entre incidencias, contenedores, barrios y datos geoespaciales.
- ☑ La auditoría es esencial: se necesita registrar quién realizó cada acción y cuándo.
- ☑ El cumplimiento de RGPD exige un control riguroso sobr la integridad y trazabilidad de los datos personales.
- ☑ En conjunto, estas necesidades encajan de forma natural con las capacidad de una base de datos relacional.

5. Consecuencias
Positivas
- ☑ Se garantiza la integridad referencial entre entidades críticas
- ☑ Las transacciones ACID permiten mantener consistencia fuerte en operaciones sensibles.
- ☑ SQL facilita la ejecucion de consultas complejas y optimizadas.
- ☑ La auditoría y trazabilidad se implementan de forma más directa
- ☑ El modelo relacional simplifica el cumplimiento normatico en materia de datos personales.

Negativas
- ☑ La escalabilidad horizontal es más limitada que en algunos sistemas NoSQL
- ☑ Requiere un diseño inicial más limitada que en algunos sistemas NoSQL.
- ☑ Cambio en la estructura pueden ser más costosos.

6. Estado
El estado es Aprobado.
La base de datos relacional será utilizada como tecnología oficial desde el inicio del proyecto.

*Tarea 3: El Diseño Detallado - Ref. 1.3.6*
*(La receta de cocina paso a paso)*
*Ya tenemos el escaparate (UI) y los cimientos (Arquitectura). Ahora vamos a detallar cómo funciona un engranaje pequeño por dentro.*
*Vuestra Misión:*
*Elegid el proceso de "Login del Conductor" y escribid su Diseño Detallado como si fuera una lista de instrucciones simples para un robot:*
*1. El conductor introduce su "ID de Empleado" y "Contraseña" en el móvil.*
*2. La aplicación móvil (Frontend) envía estos datos al Servidor (Backend).*
*3. El Servidor busca en la Base de Datos si el ID existe.*
*4. Si la contraseña es correcta, el Servidor responde: "¡Bienvenido, aquí tienes tu ruta de hoy!".*
*5. Si es incorrecta, el móvil muestra un mensaje en rojo: "Datos incorrectos, inténtalo de nuevo".*

El diseño detallado del proceso de autenticación de los conductores así como sus instrucciones claras servirán de guía para los desarrolladores durante la implementación del módulo proporcionando un proceso de fácil comprensión y altamente seguro.

1. Tabla del Proceso de autenticación del conductor

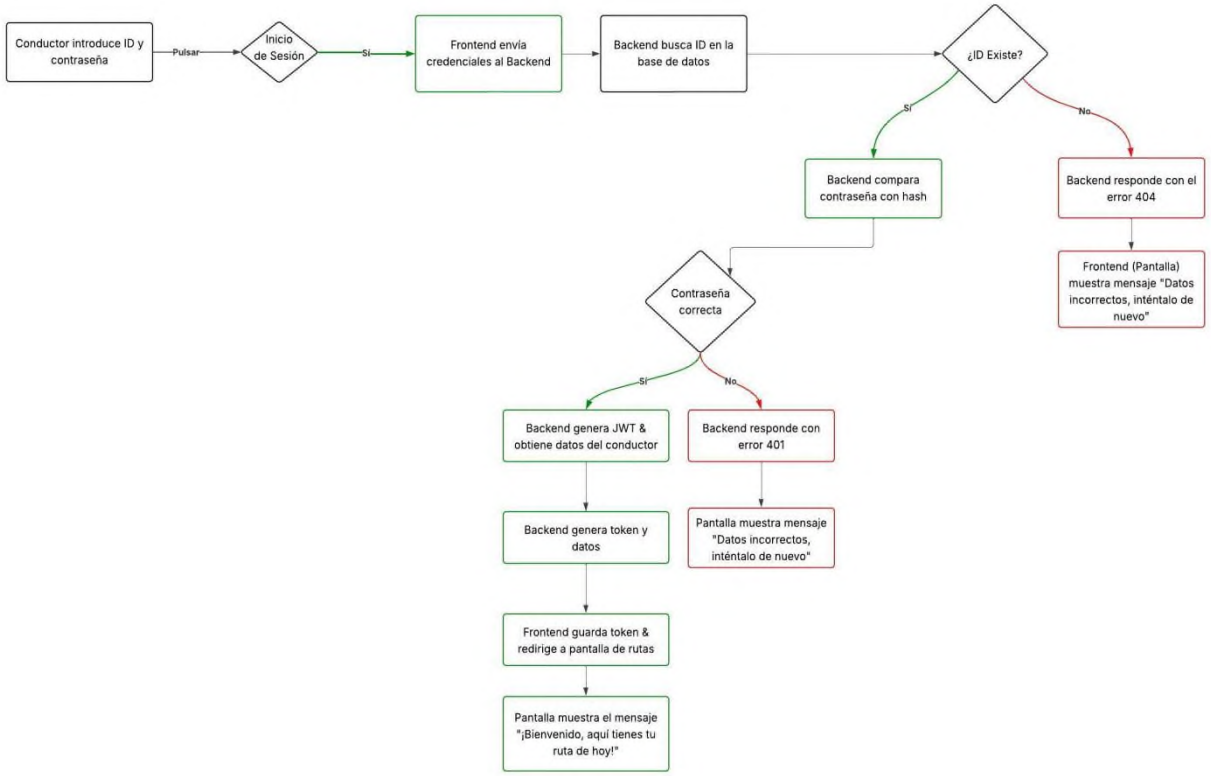
| Paso | Responsable | Acción | Descripción |
|---|---|---|---|
| 1 | Conductor | El conductor introduce sus credenciales | En la pantalla de inicio de sesión, escribe su ID de empleado y su contraseña en los campos correspondientes. |
| 2 | Conductor | El conductor pulsa el botón "Iniciar sesión" | Al pulsar el frontend recoge los datos del formulario. |
| 3 | Frontend | La aplicación móvil (Frontend) envía los datos al Servidor (Backend) | Se realiza una petición a la API REST con el ID y la contraseña en el cuerpo de la petición. |
| 4 | Backend | El servidor busca en la Base de Datos si el ID de empleado existe | El servidor ejecuta una consulta a la tabla usuarios con rol conductor filtrando por el ID recibido. |
| 5 | Backend | Si el ID no existe: el servidor responde con error 404 (usuario no encontrado) | La respuesta indica que no hay un conductor con ese ID. |
| 6 | Backend | Si el ID existe: el servidor obtiene el hash de la contraseña almacenado y lo compara con la contraseña introducida | Utiliza una función de verificación encriptada para comprobar si la contraseña es correcta. |
| 7 | Backend | Si la contraseña es incorrecta: el servidor responde con error 401 (credenciales inválidas) | La respuesta incluye un mensaje de error genérico para evitar información de seguridad. |
| 8 | Backend | Si la contraseña es correcta: el servidor genera un token JWT (JSON Web Token) que incluye el ID, rol (conductor) y fecha de expiración | El token se firma con un password cifrado y se establece un tiempo de expiración |
| 9 | Backend | El servidor responde con el token y los datos básicos del conductor (nombre, ID, ruta asignada para el día) | La respuesta con los campos: token, conductor (nombre, id), ruta (si existe). |
| 10 | Frontend | La aplicación móvil recibe la respuesta y procesa el resultado | Si la respuesta es exitosa, guarda el token en almacenamiento local (seguro) y redirige al conductor a la pantalla de ruta. Si es error, muestra un mensaje en rojo en la misma pantalla de login. |
| 11 | Frontend | El móvil muestra el mensaje correspondiente | Caso éxito: "¡Bienvenido, [nombre del conductor]! Aquí tienes tu ruta de hoy." Caso error: "Datos incorrectos, inténtalo de nuevo." |

2. Tabla de Notas de Seguridad

| Aspecto | Descripción | Medida adoptada |
|---|---|---|
| Almacenamiento de contraseñas | Las contraseñas no deben guardarse en texto plano para evitar filtraciones. | Se almacenan únicamente hashes generados con un algoritmo robusto (bcrypt) que dificulta su descifrado. |
| Verificación de contraseñas | Al iniciar sesión, el sistema debe comprobar la contraseña sin exponer el hash almacenado. | Se utiliza una función de comparación que recibe la contraseña ingresada y el hash guardado devolviendo un resultado binario |

| | | |
|---|---|---|
| | | sin mostrar datos internos. |
| Tokens de autenticación (JWT) | Una vez autenticado, el conductor recibe un token que acredita su identidad en peticiones posteriores. | El token se firma con una clave secreta, con tiempo de expiración que transmite solo por canales seguros. |
| Cifrado de comunicaciones | Los datos enviados entre el móvil y el servidor deben estar protegidos contra interceptación. | Todas las comunicaciones se realizan por HTTPS con TLS 1.2/1.3 para garantizar confidencialidad e integridad |
| Mensajes de error | Los mensajes de error no deben revelar si un identificador existe o no, para dificultar ataques de enumeración. | Se muestran mensajes genéricos como “Datos incorrectos, inténtalo de nuevo” tanto si el ID no existe como si la contraseña es errónea. |
| Protección contra fuerza bruta | Evitar que un atacante pruebe miles de contraseñas automáticamente. | Se implementa un límite de intentos fallidos por IP o por usuario y bloqueo temporal o registro de eventos sospechosos. |
| Gestión de sesiones | La sesión debe ser segura y no permitir suplantación. | El token se guarda en el lado del cliente en almacenamiento seguro. Las peticiones posteriores incluyen el token en la cabecera de autorización. |
| Auditoría de accesos | Registrar quién accede al sistema y desde dónde, por motivos de trazabilidad y detección de incidentes. | Se crea un registro (log seguro de los intentos de login con éxitos y fallos) con ID de empleado, dirección IP, timestamp y resultado. Estos logs son revisados periódicamente por el equipo de seguridad. |

3. Diagrama de Flujo del Proceso de autenticación del conductor



*Tarea 4: Técnicas de Diseño y Validación - Ref. 1.3.7*
*(La revisión de los planos antes de construir)*
*Los ingenieros nunca se fían de su primer diseño. Usan técnicas para buscar errores antes de que sea tarde. Vamos a usar la técnica de Revisión por Pares (Peer Review).*
*Vuestra Misión:*
*Simulad una revisión. Como Jefes de Proyecto, vais a escribir un pequeño párrafo de "Autocrítica" sobre el boceto que hicisteis en la Tarea 1.*
*● Ejemplo de Autocrítica: "Al hacer la revisión por pares, el equipo se dio cuenta de que en el móvil del conductor pusimos un mapa muy bonito, pero nos olvidamos de poner un botón de Emergencia/Avería del camión. Hay que añadirlo al diseño antes de programar.*

1. Revisión por pares (Peer Review) – Autocrítica del wireframe del conductor
Para asegurar que el diseño fuera realmente usable antes de pasar a desarrollo, realizamos una revisión por pares del wireframe del conductor.
En la sesión de revisión participaron un perfil senior, uno junior y la Project Manager. De este análisis, el equipo identificó que necesitamos reforzar los siguientes puntos del diseño relacionados con el conductor, que pueden afectar a su experiencia real en sus condiciones de trabajo habituales:

a) Mapa demasiado pequeño para la interacción táctil.
El mapa ocupa parte derecha de la pantalla, y la lista de las paradas parte izquierda. Sin embargo, las imágenes de los contenedores en el mapa son pequeños y difíciles de pulsar con precisión mientras se conduce o con guantes. El conductor, no puede seleccionar el contenedor para seleccionar una parada y esto puede generar errores o frustración.

Corrección: Permitir ocultar el mapa para mostrar solo la lista de paradas ampliada cuando se requiera mayor concentración.
Permitir ocultar la lista de paradas y mostrar solo el mapa aumentando el tamaño de las imágenes de los contenedores y añadir un zoom gestual más amplio para que el conductor pueda acceder desde el mapa.

b) No hay modo sin conexión (offline)
Se valorado la aplicación en las zonas urbanas con cobertura pero no el conductor puede perder cobertura o bien las rutas pueden transcurrir por zonas rurales o con escasa cobertura. En estos casos la aplicación dejaría de sincronizarse con el sistema y la base de datos.

Corrección: Implementar un modo offline que pueda almacenar localmente las acciones de al menos 1-3 días y que se vuelva a sincronizar automáticamente cuando se recupere la conexión.
Se debe incorporar a la pantalla un indicador que muestre al conductor que se encuentra offline o “sin conexión”.

c) Contraste insuficiente para conducción diurna
La pantalla que incluye la lista de paradas, el mapa y los botones que utilizará el conductor no ha sido probada a la luz del día, por lo que algunos elementos pueden resultar ilegibles. Se ha detectado que el mapa con fondo claro y marcadores de las paradas en gris pueden resultar difíciles de distinguir.

Corrección: Se propone realizar pruebas en exterior para ajustar los colores con el fin de adecuar los colores a la luz solar directa y también para que sean visibles en tardes oscuras. Igualmente aumentar el contraste de los botones y los marcadores de las paradas, así como aumentar el borde de los mismos.

*INSTRUCCIONES DE ORGANIZACIÓN PARA TU EQUIPO*
*Como responsables de vuestros 21 "empleados", os sugiero repartir el trabajo así:*
*● Trainees (Principiantes): Son los más indicados para hacer la Tarea 1 (UI/UX). Al tener menos "vicios" técnicos, pensarán más como usuarios reales de la calle. Que dibujen las pantallas.*
*● Juniors (Nivel Medio): Encargadles la Tarea 3 (Diseño Detallado). Escribir el "paso a paso" lógico les ayuda a estructurar su mente antes de programar.*
*● Seniors (Expertos): Ellos deben tomar las decisiones de la Tarea 2 (Arquitectura - ADR) y dirigir la Tarea 4 (Validación y Autocrítica) buscando los puntos débiles del diseño.*

1. Organización del trabajo por perfiles
Para aprovechar al máximo las capacidades del equipo, las tareas de esta fase se han asignado según la experiencia de cada perfil, siguiendo las siguientes directrices:

- **Trainees (principiantes)**

Responsables de la Tarea 1 (UI/UX – Wireframe del conductor).
Al no tener "vicios" técnicos, se centran en la experiencia del usuario real, dibujando pantallas accesibles e intuitivas.

- **Juniors (nivel medio)**

Encargados de la Tarea 3 (Diseño Detallado – Login del Conductor).
Redactar el paso a paso lógico les ayuda a estructurar su pensamiento antes de programar.

- **Seniors (expertos)**

Lideran la Tarea 2 (Arquitectura – ADR) y la Tarea 4 (Validación y Autocrítica). Toman las decisiones arquitectónicas y supervisan la revisión por pares para detectar puntos débiles del diseño.

Curso:  Metodología y gestión de proyectos de Ingeniería Software (IS)

Proyecto: EcoLogistics Web (Sistema Inteligente de Gestión de Residuos)

Responsable Académico: José Luis Martínez Vila

Responsable de Ejecución: Angelina Caballero (Rol:Estudiante/Project Manager)

Semana:  5 Año: 2026

# TABLA DE CONTENIDOS

*FASE 3 (Parte III): DISEÑO DE DATOS, OBJETOS Y LÓGICA* ..... 1
*Tarea 1: El Diseño de Datos y Seguridad (Ref. 1.3.8) (Nuestros archivadores digitales)* ..... 1
1. Tabla de la Ficha de Datos de Ciudadano - Seguridad ..... 1
2. Otras reglas de seguridad a tener en cuenta ..... 1
*Tarea 2: Diseño Orientado a Objetos (OO) y UML (Ref. 1.3.10 y 1.3.11)* ..... 2
1. Diseño Orientado a Objetos (OO) - Clases Contenedor y Camión ..... 2
A- Clase Contenedor (Objeto Contenedor) ..... 3
B- Clase Camión (Objeto Camión) ..... 3
C- Relación entre ambos objetos (Objeto Contenedor & Camión) ..... 3
2. Diagrama UML de Clases - Clases Contenedor y Camión ..... 3
*Tarea 3: Diseño Estructurado / Flujo Lógico (Ref. 1.3.9 y 1.3.11)* ..... 3
1. Diagrama de Flujo - Contenedor está roto ..... 4
2. Explicación del diagrama de Flujo cuando el contenedor está roto ..... 4
*INSTRUCCIONES DE ORGANIZACIÓN PARA TU EQUIPO* ..... 4
1. Asignación de tareas en la fase de diseño visual, estructura y validación ..... 4

# Semana 5 – DISEÑO DE DATOS, OBJETOS Y LÓGICA DEL SISTEMA ECOLOGISTICS WEB

*FASE 3 (Parte III): DISEÑO DE DATOS, OBJETOS Y LÓGICA*
*Proyecto: EcoLogistics Web*
*Documentación base: Puntos 1.3.8 a 1.3.11 del temario ADGG086PO.*
*Vuestro Dossier de Diseño debe completarse con las siguientes tres tareas prácticas.*
*Como Jefes de Proyecto, vuestro objetivo es que estos "planos" sean tan claros que vuestros programadores no tengan dudas.*
*Tarea 1: El Diseño de Datos y Seguridad (Ref. 1.3.8) (Nuestros archivadores digitales)*
*En EcoLogistics vamos a manejar datos sensibles de ciudadanos.*
*El diseño de datos no es solo hacer tablas, es decidir cómo protegemos esa información.*
*Vuestra Misión:*
*Elegid la entidad "Ciudadano" y cread una "Ficha de Datos" muy sencilla (como si fuera una ficha de cartulina).*
*1. Atributos: ¿Qué vamos a guardar? (Ej: Nombre, Email, Teléfono, Contraseña).*
*2. Regla de Seguridad: Escribid una regla obligatoria basada en el documento 1.3.8.*
*Por ejemplo: "La contraseña del ciudadano nunca se guardará en texto normal, se usará cifrado (hash) para cumplir con la ley de protección de datos".*

Vamos a utilizar la siguiente ficha de datos como guía para los desarrolladores para garantizar la seguridad desde la fase de diseño y evitando cualquier tipo de vulnerabilidad de seguridad futura.

1. Tabla de la Ficha de Datos de Ciudadano - Seguridad

| Atributo | Tipo | Descripción | Regla de seguridad |
|---|---|---|---|
| Identificador único (ID) | Interno | Código asignado automáticamente al registrar al ciudadano. | Es un dato interno que no se expone en la interfaz pública. |
| Contraseña | Texto (hash) | Credencial de acceso al sistema. | Regla de seguridad obligatoria. Nunca se almacena en texto plano sino bcrypt. La verificación se realiza comparando el hash, sin exponer la contraseña original. |
| Nombre completo | Texto | Nombre y apellidos del ciudadano. | Se almacena en texto plano pero acceso restringido solo a administradores |
| Correo electrónico | Texto | Dirección de contacto del ciudadano. | Solo se muestra en su perfil y en comunicaciones necesarias. |
| Fecha de registro | Fecha y hora | Momento en que el ciudadano se dio de alta. | Es un dato interno no público. Se usa para auditoría y trazabilidad; |
| Teléfono | Texto | Número de contacto. | Se trata con el mismo nivel de protección que el correo. |
| Dirección | Texto | Calle, número, ciudad, código postal. | Se almacena en texto plano.No se comparte con terceros y solo se utiliza para ubicación geográfica y envío de notificaciones. |

2. Otras reglas de seguridad a tener en cuenta

☑ Minimizar el almacenamiento de datos
Unicamente se almacenarán los datos estrictamente necesarios para el funcionamiento correcto.

(reporte de incidencias, notificaciones, autenticación).

☑ Control de accesos autorizado
Los datos personales de los ciudadanos unicamente serán visibles para el propio ciudadano y para los administradores con perfil autorizado.Los conductores no tendrán acceso a los datos de ningún ciudadano.

☑ Cumplimiento del Reglamento General de Proteción de Datos (RGPD)
(UE 2016/679) es la normativa europea, vigente desde mayo de 2018, que regula el tratamiento de datos personales de ciudadanos de la UE. Se aplica a cualquier organización (pública o privada) que maneje datos de residentes en la UE, incluso si la entidad está fuera de la UE.
Acorde a dicho reglamento se debe:
* Registrar el conseguimiento explicito del ciudadano en el momento del regístro.
* Mantener un registro de accesos a los datos personales del ciudadado con proposito de auditoría.
* Registrar que el ciudadano puede solicitar la rectificación o eliminación de sus datos y el derecho al elvido.
Los datos personales del ciudadano deben estar cifrados en todo momento mediante HTTPS.

*Tarea 2: Diseño Orientado a Objetos (OO) y UML (Ref. 1.3.10 y 1.3.11)*
*(Las piezas del Lego)*
*El software moderno se construye por "Objetos" que interactúan entre sí. Vamos a usar UML (Lenguaje Unificado de Modelado) para dibujar estas piezas. No hace falta un software profesional, podéis usar Draw.io o incluso formas en Word/PowerPoint.*
*Vuestra Misión:*
*Dibujad un Diagrama de Clases UML básico para dos piezas clave de nuestro sistema: el Camion y el Contenedor.*
*● Caja 1: Contenedor Datos (Atributos): ID, Tipo (Vidrio/Cartón), Nivel_Llenado (%). Acciones (Métodos): avisarSiEstaLleno().*
*● Caja 2: Camion Datos (Atributos): Matricula, Capacidad_Maxima. Acciones (Métodos): recogerBasura().*
*● Nota de Encapsulamiento: Poned un candado (o un signo -) al lado de "Nivel_Llenado" para indicar que es un dato protegido que no se puede cambiar por accidente.*

1. Diseño Orientado a Objetos (OO) - Clases Contenedor y Camión

En el diseño orientado a objetos (OO) se piensa en el sistema como un conjunto de objetos que representan elementos del mundo real. En este sistema cada objeto consta de:
- Atributos -> la información que guarda
- Métodos -> las acciones que puede realizar
- Reglas internas -> cómo protege sus datos (encapsulamiento)

Este tipo de diseño (atributos, métodos y reglas) ayuda a que las clases sean más faciles de enteneder, mantener y ampliar.

En Ecologistics Web el obejetivo es representar los objetos Contenedor y Camión ya que son imortantes porque forman parte del proceso de recogida de residuos. En este caso constatamos que cada uno tienen:
- atributos (sus datos)
- métodos (lo que pueden hacer)
- reglas internas que protegen su información interna (encapsulamiento).

A- Clase Contenedor (Objeto Contenedor)

- En el mundo real

Se trata de un contenedor físico que tiene un tipo, una ubicación y un nivel de llenado.

- En Software

Se trata de un objeto que guarda información sobre el contenedor y permite consutar o actualizar su estado.

  - Atributos: ID, Tipo, Ubicación, Nivel de llenado (privado), estado (normal, averiado,lleno, etc)
  - Métodos: Obtener el nivel, Actualizar el nivel, actualizar el nivel, registrar el nivel, registrar incidencia, etc.
  - Encapsulamiento: El atributo nivel de llenado está protegido (es privado). No puede ser modificado directamente sino que solo puede cambiar a través de un método que valide el dato para evitar errores y mantener la coherencia del sistema.

B- Clase Camión (Objeto Camión)

- En el mundo real

Se trata de un vehículo cuya función es recoger los contenedores y que sigue una ruta establecida con anterioridad.

- En Software

Se trata de un objeto que guarda la información del camión que permite ejecutar las acciones relacionadas con su actividad.

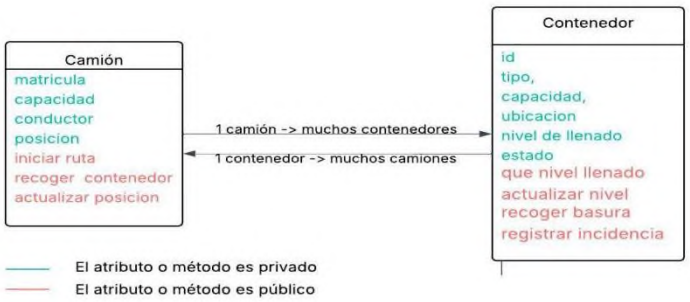
  - Atributos:Matrícula, capacidad, conductor, posición.
  - Métodos: Inciciar ruta, recoger contenedor, actualizar posición, etc.

C- Relación entre ambos objetos (Objeto Contenedor & Camión)

- En el mundo real

-Un camión interactúa con muchos contenedores: un camión -> muchos contenedores
También un contenedor puede ser recogido por distintos camiones a lo largo del tiempo es decir: un contenedor -> muchos camiones

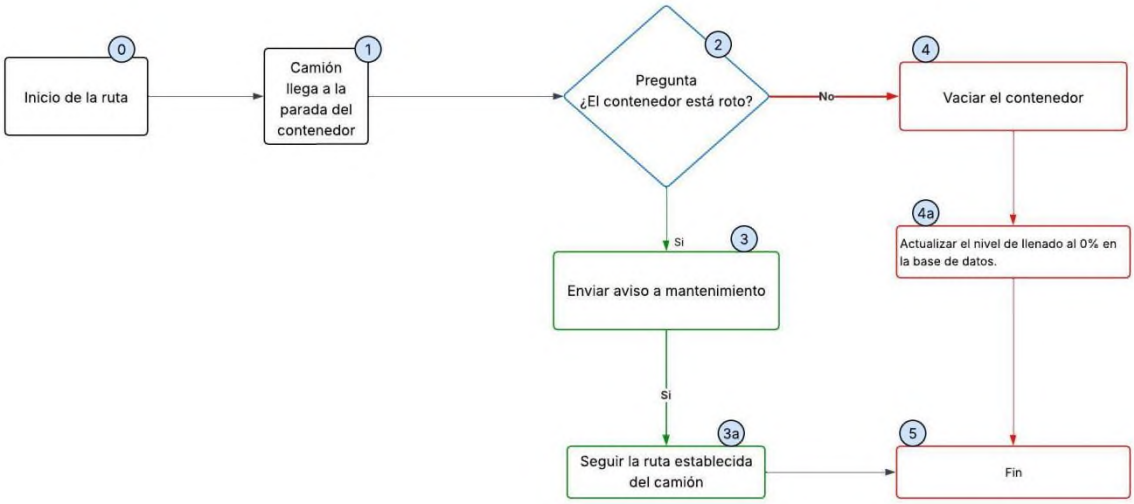
2. Diagrama UML de Clases - Clases Contenedor y Camión




*Tarea 3: Diseño Estructurado / Flujo Lógico (Ref. 1.3.9 y 1.3.11)*
*A veces, las acciones son simplemente un paso a paso con decisiones (diseño estructurado). Vamos a dibujar cómo piensa el sistema cuando un camión llega a un contenedor.*
*Vuestra Misión:*
*Cread un pequeño Diagrama de Actividad (un diagrama de flujo con flechas y rombos de decisión) para la acción "Llegada al contenedor":*
*1. Paso 1: El camión llega al contenedor.*

*2. Decisión (Rombo): ¿El contenedor está roto?*
*Si es SÍ (Flecha A): Enviar aviso a mantenimiento y seguir ruta.*
*Si es NO (Flecha B): Vaciar contenedor.*
*3. Paso 3: Actualizar nivel de llenado a 0% en la base de datos.*
*4. Fin.*

1. Diagrama de Flujo - Contenedor está roto




2. Explicación del diagrama de Flujo cuando el contenedor está roto

Paso 0: Inicio de la ruta

Paso 1: El camión llega a la parada del contenedor

Paso 2: Pregunta ¿El contenedor está roto?

Paso 3: 3a (Si) Enviar aviso a mantenimiento

Paso 3a: (Si) Seguir ruta

Paso 4: (No) Vaciar el contenedor

Paso 4a: (No) Actualizar el nivel de llenado al 0% en la base de datos.

Paso5: Fin

*INSTRUCCIONES DE ORGANIZACIÓN PARA TU EQUIPO*
*Para esta entrega, distribuye a tus 21 recursos humanos de la siguiente manera:*
*● Trainees (Principiantes): Asígnales la Tarea 1 (Diseño de Datos). Pensar en qué datos guarda un usuario y cómo proteger su contraseña es algo muy intuitivo que les ayudará a entender la privacidad web.*
*● Juniors (Nivel Medio): Encárgales la Tarea 3 (Flujo Lógico). Dibujar rombos y flechas es la mejor manera de entrenar la mente estructurada antes de escribir sentencias if/else en código.*
*● Seniors (Expertos): Ellos deben encargarse de la Tarea 2 (UML de Objetos). Requiere un poco más de abstracción técnica entender cómo interactúan el camión y el contenedor.*

1. Asignación de tareas en la fase de diseño visual, estructura y validación
(Fase 3 II)

| Perfil | Tarea asignada | Justificación |
|---|---|---|
| Trainees | Tarea 1: Diseño de Datos (atributos de la ficha del ciudadano y definición de la regla de seguridad para la contraseña) | Es una actividad muy intiutiva que no requiere conocimientos muy técnicos ni profundos. Necesitan pensar que datos guarda un usuario.La seguridad de la contraseña y la privacidad de la web es necesaria. Aunque necesita la supervisión de un junior o un senior puede ser realizada por un trainee. |
| Juniors | Tarea 3: Flujo Lógico (diagrama de actividad del proceso "Llegada al contenedor") | Elaborar diagramas como el diagrama de actividad del proceso del "llegada al contenedor" necesita de decisiones y transiciones que fortalencen el pensamiento estructurado necesario para la aplicación de código. Este tipo de lógica de flujos es necesaria para los perfiles junior y permite consolidar este tipo de aprendizaje. |
| Seniors | Tarea 2: Diagrama UML de Objetos (clases Camion y Contenedor, con atributos, métodos y encapsulamiento) | El Diagrama UML de Objetos requiere un mayor nivel de abstracción y este tipo de perfiles dominan la notación UML por lo que además pueden orientar, enseñar y acompañar al resto del equipo durante el proceso de creación. |

Curso:  Metodología y gestión de proyectos de Ingeniería Software (IS)

Proyecto: EcoLogistics Web (Sistema Inteligente de Gestión de Residuos)

Responsable Académico: José Luis Martínez Vila

Responsable de Ejecución: Angelina Caballero (Rol:Estudiante/Project Manager)

Semana:  6 Año: 2026

# TABLA DE CONTENIDOS

*FASE 4: GESTIÓN DE PROYECTO Y ESTRATEGIA* ..... 1
*Tarea 1: El "Triángulo de Hierro" (Ref. 2.1.1 y 2.1.2)* ..... 1
1. El "Triángulo de Hierro" ..... 1
*Tarea 2: Objetivos SMART (Ref. 2.2 y 2.2.1)* ..... 2
1. Criterios SMART ..... 2
A. Específicos ..... 2
B. Medible ..... 2
C. Alcanzable ..... 2
D. Relevante ..... 2
E. Temporal ..... 2
*Tarea 3: El Mapa de Interesados / Stakeholders (Ref. 2.2.3)* ..... 3
1. Tabla de Stakeholders Interesados del Proyecto Ecologistics Web ..... 3
*Tarea 4: La Estrategia de Solución y Viabilidad (Ref. 2.2.2 y 2.2.4)* ..... 3
*INSTRUCCIONES DE ORGANIZACIÓN PARA TU EQUIPO* ..... 4
1. Asignación de tareas en la fase de gestión de proyecto (Fase 4) ..... 4

# Semana 6 – FASE 4 - GESTIÓN DE PROYECTO Y ESTRATEGIA DEL SISTEMA ECOLOGISTICS WEB

*FASE 4: GESTIÓN DE PROYECTO Y ESTRATEGIA*
*Proyecto: EcoLogistics Web*
*Documentación base: Puntos 2.1.1 al 2.2.4 del temario ADGG086PO.*
*Vuestra misión como Jefes de Proyecto es entregar un documento breve (el "Acta de Constitución" o Plan Inicial) resolviendo estas cuatro tareas de manera clara y directa.*
*Tarea 1: El "Triángulo de Hierro" (Ref. 2.1.1 y 2.1.2)*
*Todo proyecto tiene tres límites inquebrantables: Tiempo, Coste y Alcance (lo que vamos a hacer). Si tocas uno, los demás se rompen.*
*Vuestra Misión:*
*Escribid un párrafo corto respondiendo a este problema:*
*El Ayuntamiento (el cliente) os acaba de llamar. Quieren que EcoLogistics Web esté lista en 3 meses en lugar de 6 (reducción de Tiempo).*
*Según el triángulo de hierro de la gestión de proyectos, ¿qué dos opciones le daréis al Ayuntamiento para que esto sea posible? (Pista: O pedís más recursos/dinero, o recortáis funcionalidades del diseño que hicimos).*

1. El "Triángulo de Hierro"

El "Triángulo de Hierro" – Carta al Ayuntamiento

De: Angelina - Proyect Manager de Ecologistics Web
Para Ayuntamiento (cliente)
Fecha:
Asunto: Reducción del plazo de entrega a 3 meses

Estimados responsables del Ayuntamiento,

En respuesta a su solicitud de reducir el plazo de entrega del proyecto Ecologistics Web de seis a tres meses, queremos trasladarles las implicaciones que este cambio supone desde la perspectiva de la gestión del proyecto. De acuerdo con el principio del Triángulo de Hierro, una reducción significativa del tiempo nos obliga necesariamente a ajustar de los dos elementos clave como son el coste o el alcance del proyecto con el fin de no comprometer la calidad del mismo.

Para cumplir con el nuevo plazo, podemos ofreceler las siguientes alternativas:

Incrementar los recursos y el presupuesto permitiendp reforzar el equipo de desarrollo y acelerar la ejecución la ejecución del proyecto. Para ello necesitaríamos incorporar desarrolladores  o servicios cloud adicionales para ejecutar más tareas en paraleleo y acelerar el desarrollo. Esta medida implicaría un aumento significativo del presupuesto, tanto en costes iniciales como en el coste total del proyecto.

Reducir el alcance funcional previsto, entregando una versión incial más acotada o Producto Mínimo Viable (MVP) que pueda completarse dentro del nuevo calendario. A esta versión con las funcionalidades imprescindibles, como el resgistro de incidencias, la gestión básica de rutas y el proceso de autenticación le podría seguir una versión más completa. Las capacidades más avanzades con los módulos de análisis, las herramientas de planificación manual y otras mejoras previstas quedarían programadas para una segunda fase, una vez estuviera operativa la primera versión.

Ambas opciones permiten mantener la calidad final del producto dentro del plazo solicitado.

Quedamos a su disposición para analizar conjutamente cuál de estas alternativas se ajusta mejor a sus prioridades.

Atentamente

El Equipo de Proyecto de Ecologistics Web

*Tarea 2: Objetivos SMART (Ref. 2.2 y 2.2.1)*
*El cliente dice: "Quiero que la aplicación mejore la limpieza de la ciudad". Eso es un deseo, no un objetivo de ingeniería. Los objetivos deben ser SMART (Específicos, Medibles, Alcanzables, Relevantes y con un Tiempo definido).*
*Vuestra Misión:*
*Transformad ese "deseo" en dos objetivos SMART para EcoLogistics.*
*Ejemplo de formato: "Reducir (qué) en un 20% (cuánto) utilizando las rutas óptimas del sistema, en un plazo de 6 meses (cuándo) tras el lanzamiento".*

1. Criterios SMART

**A. Específicos**
- Reducir en un 30% el tiempo de respuesta ante contenedores llenos mediante el uso de rutas optimizadas y notificaciones automáticas a los conductores, alcanzándolo en un plazo máximo de seis meses tras el lanzamiento de la plataforma.

- Incrementar en un 40% la proporción de incidencias correctamente reportadas gracias a un formulario más simple y a su integración con el mapa interactivo, logrando este resultado durante el primer año de operación continua.

**B. Medible**
- Disminuir en un 30% el tiempo medio que transcurre desde que un contenedor supera el 75% de llenado hasta que es vaciado.
- Incrementar en un 30% el número de incidencias válidas registradas por la ciudadanía (contenedor lleno, dañado, etc.), tomando como referencia el volumen de reportes correctos del periodo previo al lanzamiento de la plataforma.

**C. Alcanzable**
- Gracias al uso del algoritmo de optimización de rutas y al envío automático de avisos a los conductores, la reducción del tiempo de respuesta es viable dentro de los recursos y capacidades del sistema.
- Mediante la simplificación del formulario de incidencias y su integración con el mapa interactivo, el aumento de reportes válidos es factible sin requerir cambios estructurales ni inversiones adicionales.

**D. Relevante**
- Impacta directamente en la limpieza urbana y en la satisfacción de la ciudadanía, al mejorar la rapidez y eficacia del servicio de recogida.
- Refuerza la participación ciudadana y la detección temprana de incidencias, permitiendo actuar antes y con mayor precisión sobre los problemas del entorno urbano.

**E. Temporal**
- En los 6 meses siguientes al lanzamiento de la plataforma, para los objetivos relacionados con la reducción del tiempo de respuesta.
- En el primer año de operación continua, para los objetivos vinculados al incremento de incidencias válidas registradas.

*Tarea 3: El Mapa de Interesados / Stakeholders (Ref. 2.2.3)*
*Un "Stakeholder" es cualquier persona a la que le importe o le afecte nuestro proyecto. Si nos olvidamos de uno importante, el proyecto fracasará (ej. si hacemos la app sin consultar a los conductores de los camiones).*
*Vuestra Misión:*
*Haced una tabla sencilla dividiendo a los interesados en Internos (los que fabrican la app) y Externos (los que la usan o pagan).*
*• Internos: (Ej: Equipo de programadores, Jefe de proyecto).*
*• Externos: (Ej: Ciudadanos, Conductores, Alcalde).*
*• Añadid un "Interés principal" a cada uno. (Ej: El conductor quiere que la app sea fácil de leer mientras conduce).*

1. Tabla de Stakeholders Interesados del Proyecto Ecologistics Web

| Stakeholder | Tipo | Interés |
|---|---|---|
| Jefe/a de Proyecto | Interno | asegurarse de que el trabajo se entregue a tiempo, dentro del presupuesto y con la calidad acordada. |
| Equipo de Frontend (UI/UX) | Interno | diseñar una interfaz que resulte clara, accesible y agradable de usar. |
| Equipo de Backend (API) | Interno | construir una API REST sólida, segura y con buen rendimiento. |
| Equipo de Base de Datos y DevOps | Interno | definir un modelo eficiente, mejorar las consultas y mantener la infraestructura y los despliegues en buen estado |
| Equipo de QA Testing | Interno | realizar pruebas de todo tipo para garantizar que la aplicación sea estable y segura. |
| Equipo de Documentación | Interno | elaborar guías y manuales que sean fáciles de entender para usuarios y técnicos. |
| Ayuntamiento (cliente) | Externo | optimizar la gestión de residuos y reducir los costes del servicio. |
| Ciudadanos | Externo | comunicar incidencias rápidamente y encontrar contenedores sin complicaciones. |
| Conductores de camiones | Externo | aplicación que se lea bien durante la ruta y que les proponga recorridos eficientes. |
| Administrador /gestor de flota | Externo | tener el control sobre rutas, conductores, contenedores y estadísticas operativas. |
| Personal de mantenimiento | Externo | recibir avisos claros cuando un contenedor está dañado. |
| Policía Local | Externo | asegurarse de que los contenedores no generen riesgos ni bloqueen la circulación. |
| Servicios urbanos (limpieza viaria) | Externo | quiere mejorar la percepción ciudadana del servicio y disminuir el número de quejas. |
| Alcaldía / Concejales | Externo | Mejorar la imagen de la ciudad y reducir quejas. |

*Tarea 4: La Estrategia de Solución y Viabilidad (Ref. 2.2.2 y 2.2.4)*
*Antes de empezar a programar como locos, tenemos que justificar por qué nuestra solución es viable (posible de hacer) y qué especificaciones básicas tendrá.*
*Vuestra Misión:*
*Escribid tres frases, explicando con palabras de estar por casa, las tres viabilidades de EcoLogistics:*
*1. Viabilidad Técnica: "¿Tenemos la tecnología para hacer un mapa de contenedores?" (Sí, usando Google Maps o similar).*
*2. Viabilidad Económica: "¿Es más barato hacer esta web que comprar camiones nuevos?" (Justificadlo brevemente).*
*3. Viabilidad Operativa: "¿Sabrán usarla los ciudadanos?" (Sí, porque la haremos accesible y sencilla).*

1. Viabilidad Técnica
El proyecto es técnicamente viable porque contamos con la tecnologías consolidadas y gratuitas como Google Maps para la cartografía (rutas, ubicación de los

contenedores) y JavaScript con React y Node.js para el desarrollo, que permiten construir la solución sin necesidad de crear los componentes desde cero.

2. Viabilidad Económica
Desde un punto de vista económico, resulta más rentable desarrollar la plataforma que invertir en renovar la flota, dado que el propio software se amortiza gracias a la optimización de rutas, y que reducirán tanto el gasto en combustible como en horas de trabajo, mejorando el ROI. Además la plataforma se apoya en herramientas cloud de bajo coste.

3.Viabilidad Operativa
Operativamente, también es viable porque los ciudadanos podrán utilizar la plataforma sin dificultad gracias a una interfaz muy simple, con elementos claros, textos grandes y familiares, similar a aplicaciones que ya conocen complementada con pequeños tutoriales accesibles para facilitar su uso con rapidez. Además, realizaremos pruebas de usabilidad con usuarios reales, incluyendo personas que puedan tener dificultades de acceso a este tipo de aplicaciones, antes de su lanzamiento.

*INSTRUCCIONES DE ORGANIZACIÓN PARA TU EQUIPO*
*Como responsables, no tenéis que hacer todo esto solos. Repartid el trabajo entre vuestros 21 "empleados" de esta forma:*
*• Trainees (Principiantes): Son perfectos para la Tarea 3 (Stakeholders). Cualquiera puede pensar de forma lógica en quién va a usar la aplicación de basura en su ciudad sin necesidad de saber programar.*
*• Juniors (Nivel Medio): Encargadles la Tarea 2 (Objetivos SMART). Les ayudará a aterrizar las ideas vagas en metas matemáticas y medibles.*
*• Seniors (Expertos): Ellos deben enfrentarse al cliente en la Tarea 1 (El Triángulo de Hierro) y definir la Tarea 4 (Viabilidad Técnica), ya que requieren más experiencia para saber qué es posible desarrollar y qué no.*
*Formato de entrega: Un documento de 1 o 2 páginas en Word/PDF respondiendo a estas 4 tareas de forma directa.*

1. Asignación de tareas en la fase de gestión de proyecto (Fase 4)

| Perfil | Tarea asignada | Justificación |
|---|---|---|
| Trainees | Tarea 3: Mapa de Interesados (Stakeholders) | Sin saber programar se puede identificar facilmente las personas que van a utilizar la aplicación: conductores, ciudadanos, admnistrador, etc. |
| Juniors | Tarea 2: Objetivos SMART | Les permite transformar ideas generales en ojetivos concretos y medibles y así fortalecen su capacidad para analizar las cosas de manera más precisa. |
| Seniors | Tarea 1: Triángulo de Hierro y Tarea 4: Viabilidad técnica, económica y operativa | Requieren suficiente experiencia como para negociar con el cliente usando el triangulo de hierro, valorar qué partes del producto son realmente viables desde el punto de vista técnico y explicar con claridad los costes y la operatividad de cada opción. |

Curso:  Metodología y gestión de proyectos de Ingeniería Software (IS)

Proyecto: EcoLogistics Web (Sistema Inteligente de Gestión de Residuos)

Responsable Académico: José Luis Martínez Vila

Responsable de Ejecución: Angelina Caballero (Rol:Estudiante/Project Manager)

Semana:  7 Año: 2026

# TABLA DE CONTENIDOS

*PLANIFICACIÓN Y SEGUIMIENTO DEL PROYECTO* ..... 1
1. Registro de Ciudadanos - Lista de 5 tareas secuenciales con dependencias Fin-a-Inicio (FI): ..... 1
*Tarea 2: El Diagrama de Gantt (Ref. 2.4.3)* ..... 1
1. Diagrama de Gantt ..... 2
*Tarea 3: Los Hitos y sus Fechas Límite (Ref. 2.4.4)* ..... 2
1. Hitos y Fechas Límite de Ecologistics Web ..... 2
Hito 1: Backend y Base de Datos operativos con pruebas de seguridad superadas 2
Hito 2: Interfaz de Usuario Integrada y Funcionalidad Principal Implementada .. 2
Hito 3: Sistema Completo Validado y Documentación Oficial Entregada ..... 3
*Tarea 4: La Documentación Técnica Viva (Ref. 2.4.5)* ..... 3
1. ADR — Uso de Tablero Visual para la Gestión de Tareas ..... 3
*INSTRUCCIONES DE ORGANIZACIÓN PARA TU EQUIPO (20-21 Personas)* ..... 4
1. Asignación de tareas en la fase de planificación y seguimiento ..... 4

# Semana 7 – FASE 4 - PLANIFICACIÓN Y SEGUIMIENTO DEL PROYECTO DEL SISTEMA ECOLOGISTICS WEB

*PLANIFICACIÓN Y SEGUIMIENTO DEL PROYECTO*
*Proyecto: EcoLogistics Web*
*Documentación base: Puntos 2.4.1 a 2.4.5 del temario ADGG086PO.*
*Vuestra misión como Jefes de Proyecto es organizar cuándo y en qué orden vamos a construir el sistema. Debéis entregar un plan visual y documentado compuesto por las siguientes 4 tareas:*
*Tarea 1: Ordenar y Relacionar Tareas (Ref. 2.4.1 y 2.4.2)*
*(No podemos poner el tejado sin los cimientos)*
*En el desarrollo de software, hay tareas que bloquean a otras. Como vimos en la teoría, esto se llama Dependencia Fin-a-Inicio (FI). Para nosotros, la seguridad y la infraestructura van primero. No hay atajos.*
*Vuestra Misión:*
*Escribid una lista de 5 tareas secuenciales para crear el "Registro de Ciudadanos", explicando por qué una debe terminar obligatoriamente antes de empezar la siguiente.*
*• Ejemplo paso a paso:*
*1. Tarea A: Configurar la Base de Datos con datos cifrados.*
*2. Tarea B: Crear la pantalla web de registro. (Depende de A: No podemos guardar usuarios si la base de datos segura no existe).*

1. Registro de Ciudadanos - Lista de 5 tareas secuenciales con dependencias Fin-a-Inicio (FI):

Paso 1- Tarea A: Configurar la base de datos segura (PostgreSQL con cifrado)
No depende de nada previo porque es el cimiento técnico sobre el que se apoyará todo el sistema.

Paso 2- Tarea B: Crear la tabla usuarios con los campos necesarios (id, nombre, email, contraseña hash, etc.).
Depende de la tarea A: Depende de la base de datos ya configurada, ya que no se puede crear ninguna tabla si el sistema de almacenamiento aún no está operativo y seguro.

Paso 3- Tarea C: Implementar el endpoint de registro en el backend (API REST que reciba datos y los inserte en la tabla).
Depende de la tarea B: Depende de la tabla de usuarios, porque este servicio necesita que la estructura exista para poder guardar los datos que reciba.

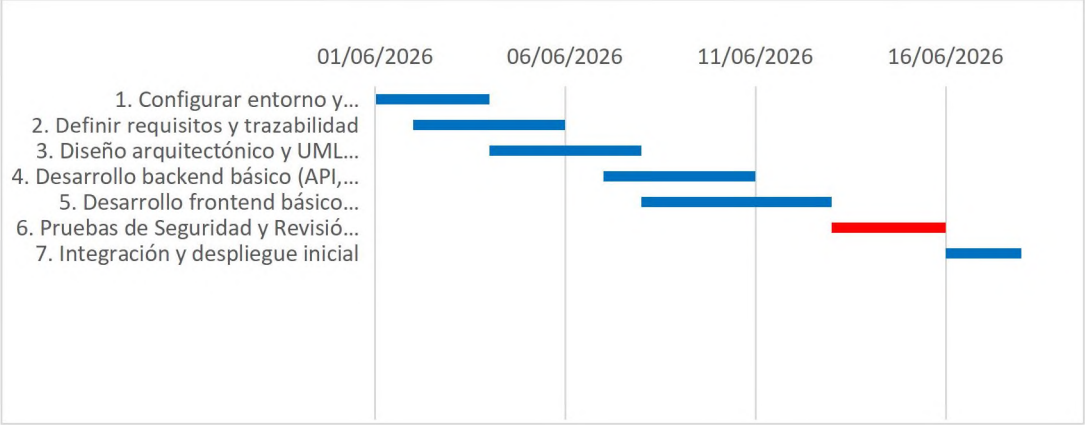
Paso 4 - Tarea D: Desarrollar la pantilla de registro en el frontend (formulario de recogida de datos, etc.)
Depende de la tarea C: Depende del endpoint funcionando, ya que la pantalla debe enviar la información del formulario a un servicio real del backend.

Paso 5. Tarea E: Realizar pruebas de integración y seguridad del flujo completo
Depende de la tarea D: Depende de que todo el recorrido esté montado (pantalla, endpoint, y base de datos) para poder comprobar que los datos viajan bien y se almacenan correctamente sin que haya fallos de seguridad.

*Tarea 2: El Diagrama de Gantt (Ref. 2.4.3)*
*(El mapa del tiempo).*
*El Diagrama de Gantt es un calendario visual que nos muestra cuánto dura cada tarea.*

*Vuestra Misión:*
*Dibujad un Diagrama de Gantt muy sencillo (podéis usar un Excel pintando celdas de colores, o dibujarlo en una pizarra y sacarle una foto) para el primer mes de trabajo.*
*• Regla de Seguridad Innegociable: Debéis incluir obligatoriamente una barra de tiempo de al menos 3 días llamada "Pruebas de Seguridad y Revisión de Código".*
*Si el cliente presiona para ir más rápido, recortamos funcionalidades (alcance), pero nunca recortamos el tiempo de las pruebas.*

1. Diagrama de Gantt




*Tarea 3: Los Hitos y sus Fechas Límite (Ref. 2.4.4)*
*(Nuestras banderas de control)*
*Un "Hito" (Milestone) no es una tarea en la que se trabaja; es un momento de celebración y control. Es un punto en el calendario donde decimos: "¡Fase completada y verificada!".*
*Vuestra Misión:*
*Definid 3 Hitos principales para EcoLogistics Web. Un hito no debe llamarse "Terminar código", debe ser un entregable de calidad.*
*• Ejemplo de Hito: "Hito 1: Base de Datos Desplegada y Auditoría de Seguridad Superada (Fecha límite: Semana 3)"*

1. Hitos y Fechas Límite de Ecologistics Web

A continuación se definen ñps tres hitos principales del proyecto EcoLogistics Web. Cada hito constituye un punto de control verificable asociado a un entregable de calidad, y no una tarea operativa. Las fechas límite se han establecido conforme al cronograma del primer mes de trabajo (3 semanas) y respetan la prioridad absoluta de los requisitos de seguridad definidos para el proyecto.

Hito 1: Backend y Base de Datos operativos con pruebas de seguridad superadas
Fecha límite: Semana 3
La API REST y el sistema de gestión de la base de datos relacional se encuentran plenamente operativos. Asimismo se han completado satisfactoriamente las de integración correspondientes (Jest & Supertest) y los análisis de seguridad mendiante OWASP ZAP, confirmando la ausencia de incidencias críticas.

Hito 2: Interfaz de Usuario Integrada y Funcionalidad Principal Implementada
Fecha límite: Semana 6
Las cistas destinadas a ciudadanos y conductores se encuentran desarrolladas e integradas con los servicios del backend. Las funcionalidades principales (incluyendo el reporte de incidencias, la consulta de contenedores y la gestión de rutas) han superado las pruebas end-to-end ejecutadas con Cypress.

Hito 3: Sistema Completo Validado y Documentación Oficial Entregada
Fecha límite: Semana 10
El sistema ha demostrado un comportamiento estable bajo pruebas de carga con 5.000 usuarios concurrentes. La documentación técnica y de usuario ha sido finalizada y revisada, y el repositorio oficial del proyecto contiene la versión definitiva del código fuente.

*Tarea 4: La Documentación Técnica Viva (Ref. 2.4.5)*
*(El diario de a bordo)*
*Si mañana hay un problema de seguridad o se incorpora un programador nuevo, nadie tiene tiempo de leer correos antiguos. Necesitamos un documento "vivo" que explique por qué tomamos ciertas decisiones hoy.*
*Vuestra Misión:*
*Redactad un pequeño Registro de Decisión (ADR) sobre cómo vais a organizar el trabajo.*
*• Título: Uso de Tablero Visual.*
*• Decisión: Vamos a usar Trello (o Jira) para ver qué tareas están "Por hacer", "En progreso" y "Terminadas".*
*• Justificación: Para que cualquier persona del equipo (o yo como profesor) pueda ver el estado del proyecto en 10 segundos sin tener que preguntar.*

1. ADR — Uso de Tablero Visual para la Gestión de Tareas

A continuación se expone un Registro de Decisión de Arquitectónica (ADR) relativo a la adopción de un tablero visual como herramienta central para la organización y el seguimiento del trabajo. Este documento, concebiendo como un recurso dinámico y permanentemente actualizado, facilita que cualquier integrante del equipo pueda identificar el estado del proyecto de manera inmediata, sin necesidad de solicitar información adicional:

| | |
|---|---|
| Título | Uso de Tablero Visual (Jira) para el seguimiento del proyecto |
| Contexto | El equipo compuesto por 21 integrantes, requiere un mecanismo de seguimiento que proporcione visibilidadinmediata sobre el estado de las tareas: Tareas pendientes, en ejecución, en revisión o finalizadas.<br>La ausencia de esta claridad genera retrasos, duplicidades y un incremento innecesario de reuniones de coordinación.<br>También, es imprescindibles que se pueda consultar el avance del proyecto de forma autónoma y en un tiempo inferior a 10 segundos. |
| Decisión | Se adopta un tablero Kanban en Jira (herramienta definida previamente en el apartado 4.2) estructurado con las columnas:<br>Por hacer, En Progreso, En Revisión y Terminado. Cada elemento de trabajo se representará mediante un tarjeta con descripció, responsable, prioridad y estimación.<br>Se incorpora además la columna En pruebas de seguridad para garantizar que ninguna tarea sea considerada finalizada sin superar los controles de seguridad obligatorios. |
| Justificación | • Transparencia operativa: El tablero permite conocer el estado del proyecto de forma inmediata, sin necesidad de solicitar información adicional.<br>• Formento de la autonomía: Los miembros del equipo pueden identificar su siguiente tarea y detectar bloqueos sin intervención |

externa.
• Flexibilidad del flujo de trabajo: La estructura del tablero puede ajustarse según las necesidades del sprint o del proyecto.
• Trazabilidad técnica: La integración nativa entre Jira y GitHub facilita la vinculación de commits y pull requets con las tareas correspondientes.
• Cumplimiento de los requisitos de seguridad: La columna específica para pruebas de seguridad asegura que el código no avance a producción sin la validación pertinente.

Alternativas consideradas

• Trello: Herramienta sencilla pero insuficiente para las necesidades del proyecto, especialmente por ña falta de integración nativa con GitHub y capacidades avanzadas de reporting.
• Excel compartido: Demasiado estático, no refleja el estao en tiempo real y requiere actualización manual.
• Tablero físico (pizarra): no accesible para trabajo remoto ni para el Proyect Manager si no está en la misma sala.

Estado

Aprobado.
El tablero de Jira se utilizará desde el inicio del proyecto como herramienta oficial de seguimiento.

*INSTRUCCIONES DE ORGANIZACIÓN PARA TU EQUIPO (20-21 Personas)*
*Como Jefes de Proyecto, vais a distribuir este trabajo de planificación sin dejar a nadie atrás:*
*• Trainees (Principiantes - 6 personas): Encargadles la Tarea 3 (Hitos). Pensar en cuáles son los momentos clave de entrega (ej. "Tener la app móvil lista para probar") es muy intuitivo y les ayuda a ver el proyecto a vista de pájaro.*
*• Juniors (Nivel Medio - 10 personas): Asignadles la Tarea 1 (Relación de tareas) y la Tarea 4 (Documentación). Obligarles a pensar qué va antes y qué va después entrena su lógica de programación.*
*• Seniors (Expertos - 5 personas): Ellos deben encargarse de la Tarea 2 (Diagrama de Gantt). Calcular tiempos reales y asegurar que se incluyen los márgenes necesarios para infraestructura y seguridad requiere la experiencia que ellos ya tienen.*
*Formato de entrega: Añadid estas 4 tareas a vuestro dossier del proyecto en un formato limpio y claro.*

1. Asignación de tareas en la fase de planificación y seguimiento
(Fase 5 – puntos 2.4.1 a 2.4.5)

| Perfil | Tarea asignada | Justificación |
|---|---|---|
| Trainees | Tarea 3: Definir los Hitos (momentos clave de entrega) | Pensar en hitos tangibles como disponer de una base de datos protegida o contar con una versión móvil lista para ser evaluada, permite obtener una visión global del proyecto y comprender su avance de un sólo vistazo. |
| Juniors | Tarea 1: Relacionar tareas (dependencias) y Tarea 4: Documentación técnica viva (ADR del tablero visual) | Fomerta la capacidad de documentar las decisiones fortalece sus competencias profesionales, mientras que |

| Perfil | Tarea asignada | Justificación |
|---|---|---|
| Seniors | Tarea 2: Diagrama de Gantt (calcular tiempos reales, incluir pruebas de seguridad) | obliga a establecer un orden lógico entre las tareas desarolla el razonamiento estructurado propio de la programación.<br><br>La correcta gestión de solapamientos y margenes en infrastructura y seguridad exige aplicar la experiencia previa que ya poseen, y que solo así se pueden tomar las decisiones ajustadas a las necesidades. |